
第九章：聚类

大纲

- 聚类任务

- 性能度量

- 距离计算

- K均值算法

监督学习 vs 无监督学习

监督学习

数据: (x, y)

x 是数据, y 是标签

目标: 学习一个函数将 x 映射到 y

例子: 分类、回归、目标检测等



→ Cat

例子: 分类

监督学习 vs 无监督学习

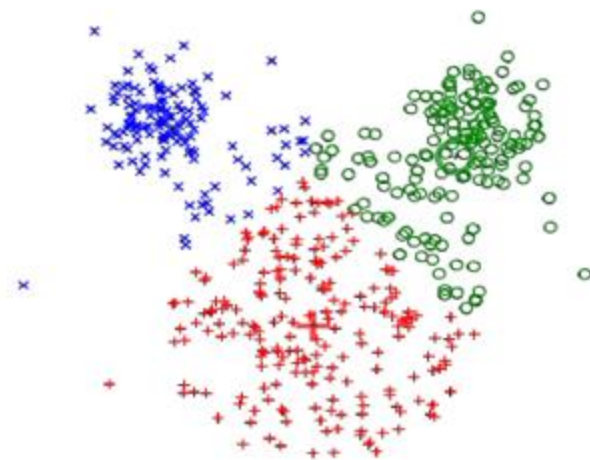
无监督学习

数据: X

只有数据, 没有标签!

目标: 学习数据的某些潜在隐藏结构

例子: 聚类、特征学习、降维等



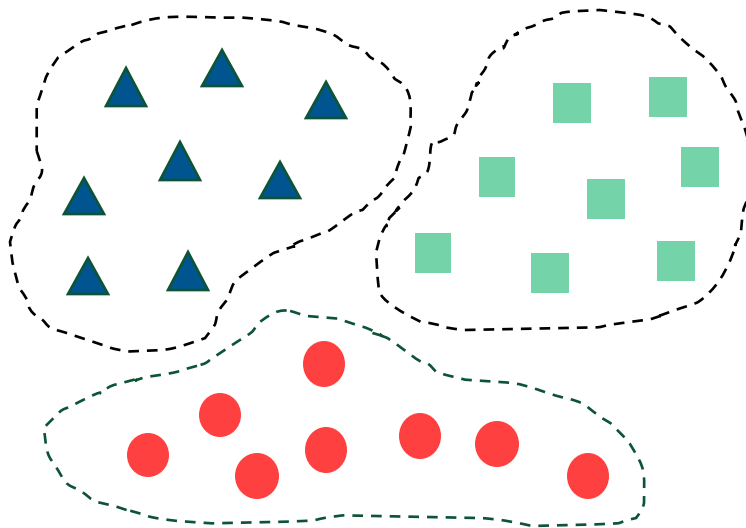
例子: 聚类

聚类任务

□ 目标

将数据集中的所有样本划分为若干个通常不相交子集（簇，cluster），每个子集对应着潜在的(未知的)类别（深色瓜/浅色瓜、有籽瓜/无籽瓜...）

- 既可以作为一个单独过程（用于找寻数据内在的分布结构）
- 也可作为分类等其他学习任务的前驱过程



聚类任务

□ 聚类(无监督学习)

假设包含 m 个无标记样本的样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$

- 其中, $\mathbf{x}_i = (\mathbf{x}_{i1}, \mathbf{x}_{i2}, \dots, \mathbf{x}_{in}) \in R^n$: 包含 n 维特征向量 (有 n 个属性)

可以划分成 k 个不相交的簇 $\{C_l | l = 1, 2, \dots, k\}$

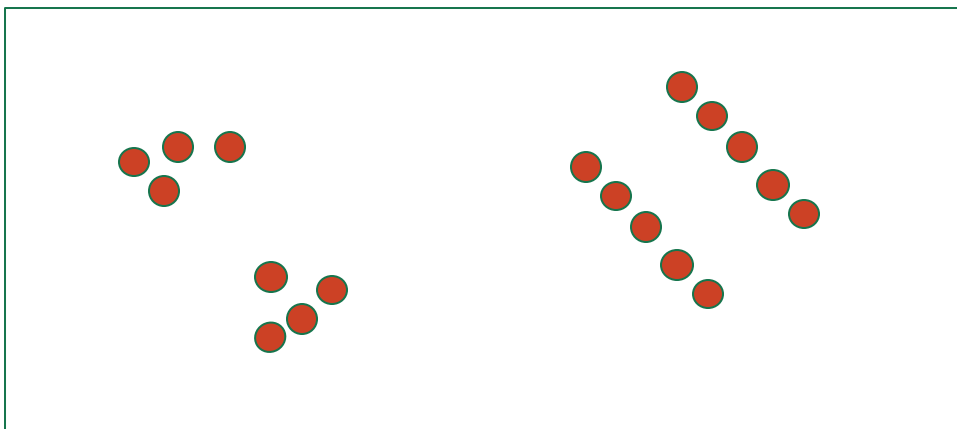
- $C_{l'} \cap_{l' \neq l} C_l = \emptyset$
- $\cup_{l=1}^k C_l = D$

样本 $\mathbf{x}_i$ 的聚类结果可以用簇标记 λ_j 进行表示, 即 $\mathbf{x}_i \in C_{\lambda_j}$

所有样本的簇标记向量 $\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_m)$

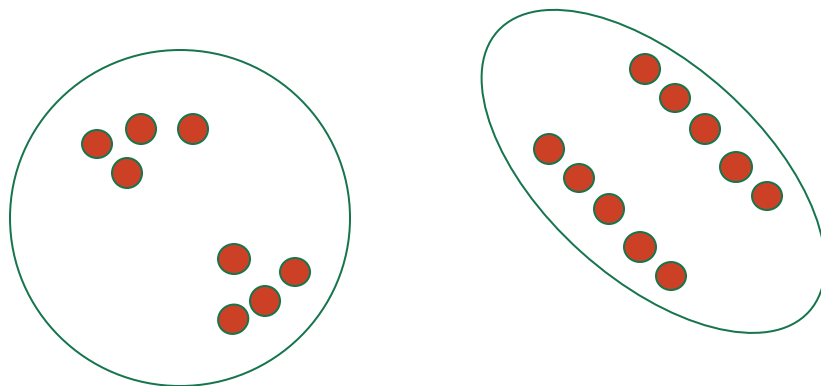
聚类例子

- 基本思想：将相似的实例聚集在一起
- 例子：2D 点模式



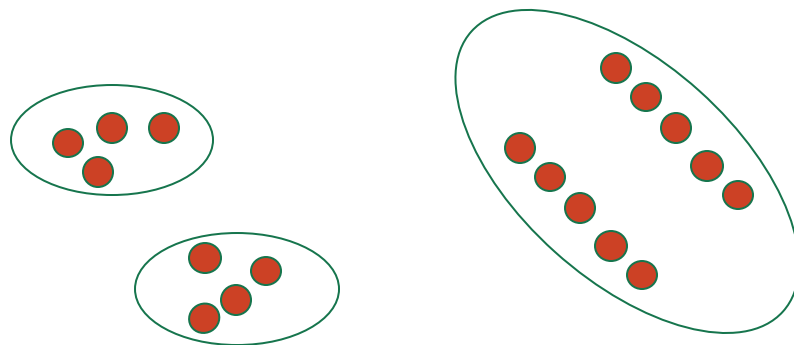
聚类例子

- 基本思想：将相似的实例聚集在一起
- 例子：2D 点模式



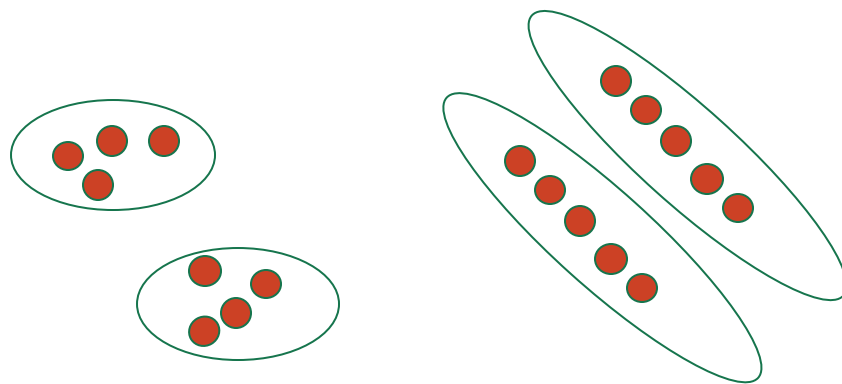
聚类例子

- 基本思想：将相似的实例聚集在一起
- 例子：2D 点模式



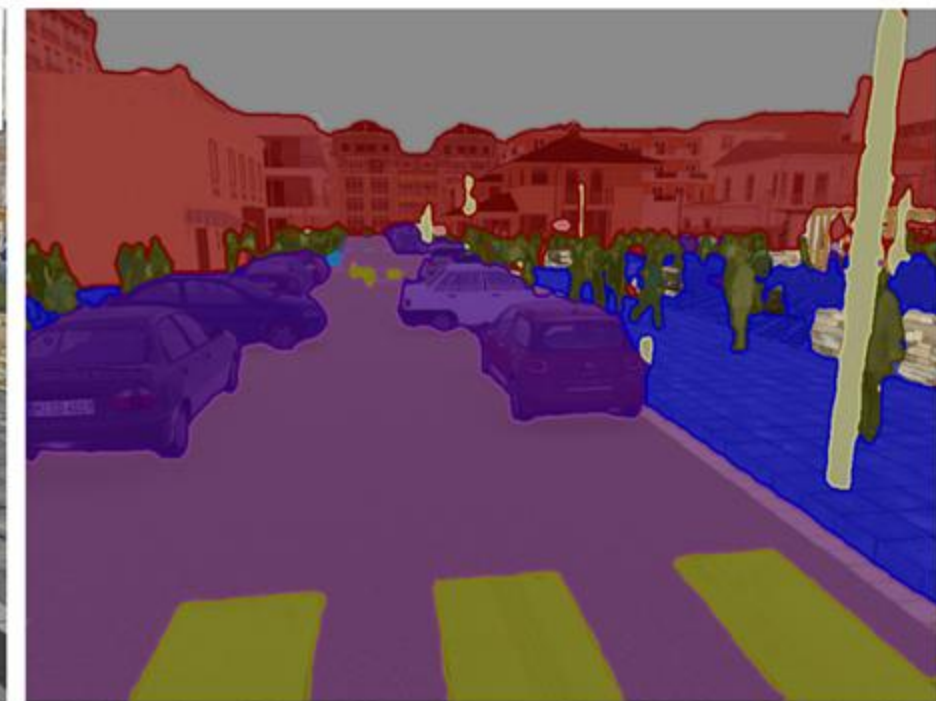
聚类例子

- 基本思想：将相似的实例聚集在一起
- 例子：2D 点模式



聚类应用例子

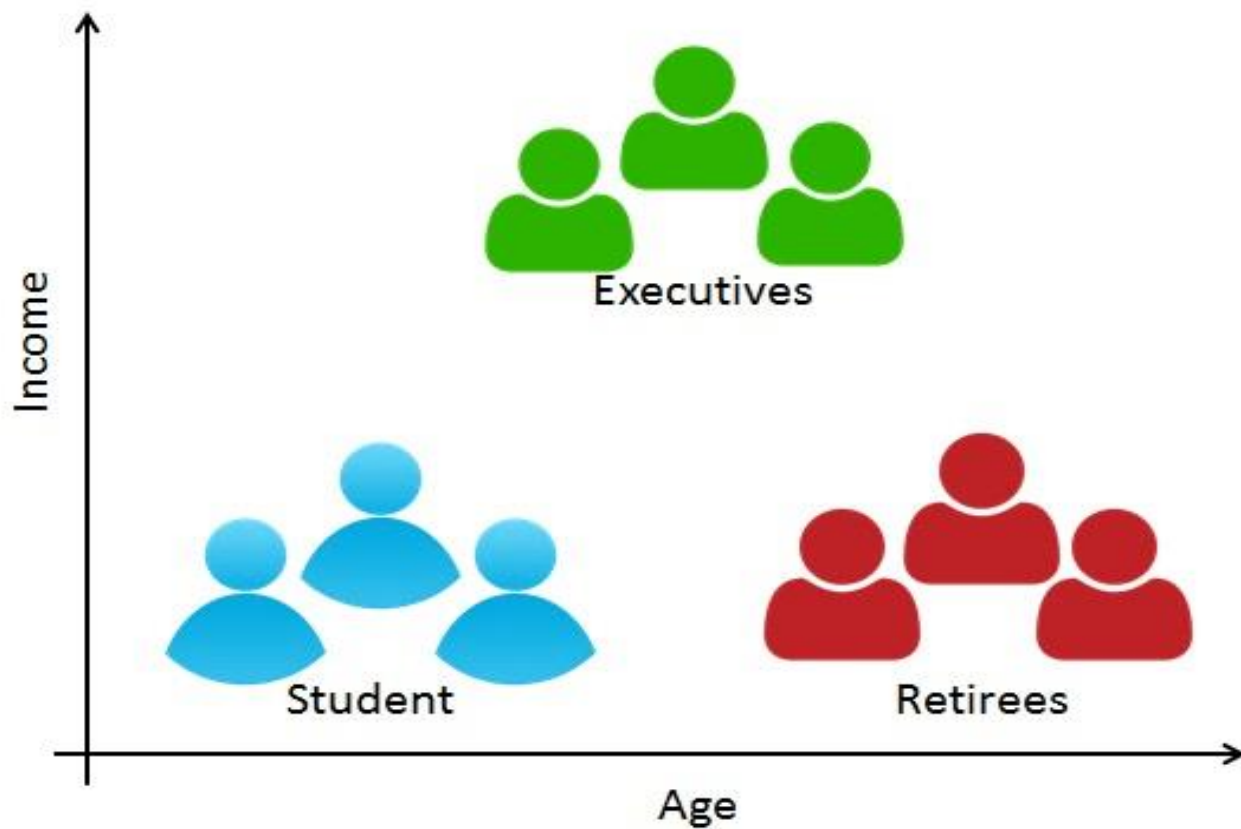
- 图像分割



■ Sky ■ Building ■ Road ■ Sidewalk ■ Fence ■ Vegetation ■ Pole ■ Car ■ Sign ■ Pedestrian ■ Cyclist

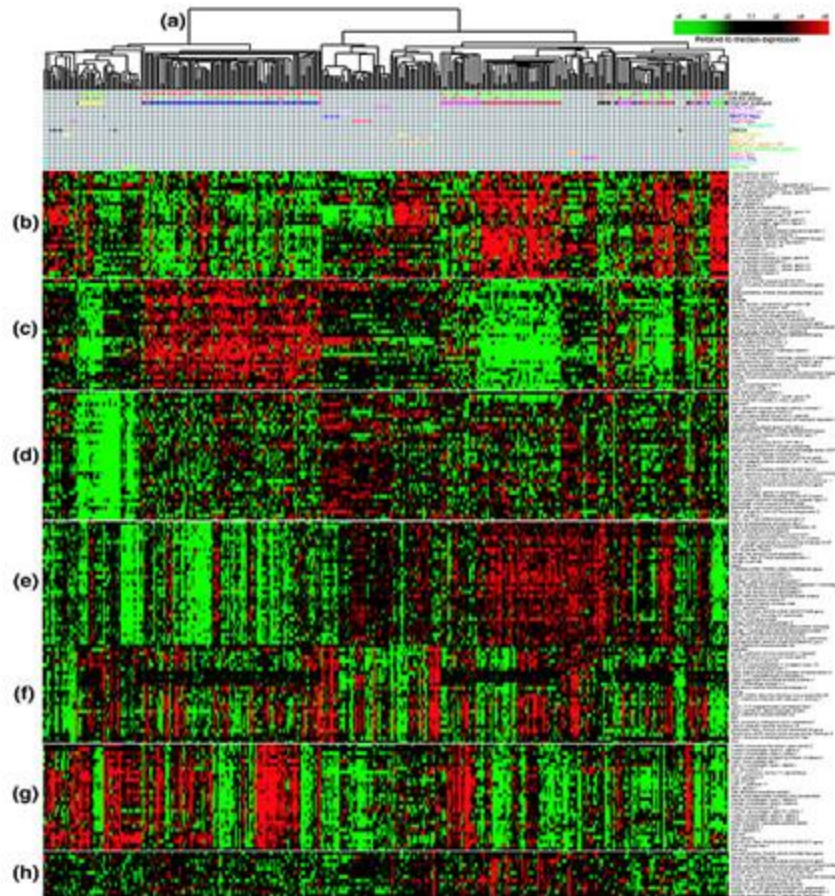
聚类应用例子

- 客户细分



聚类应用例子

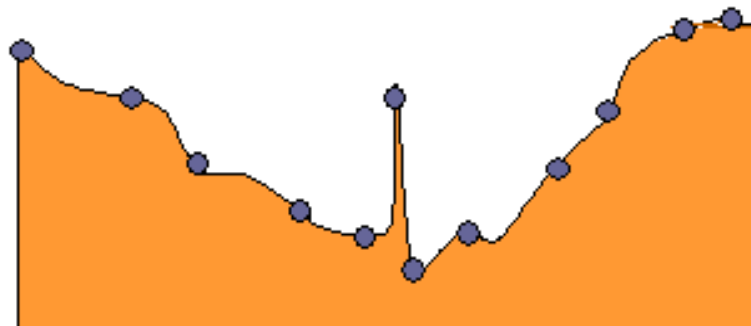
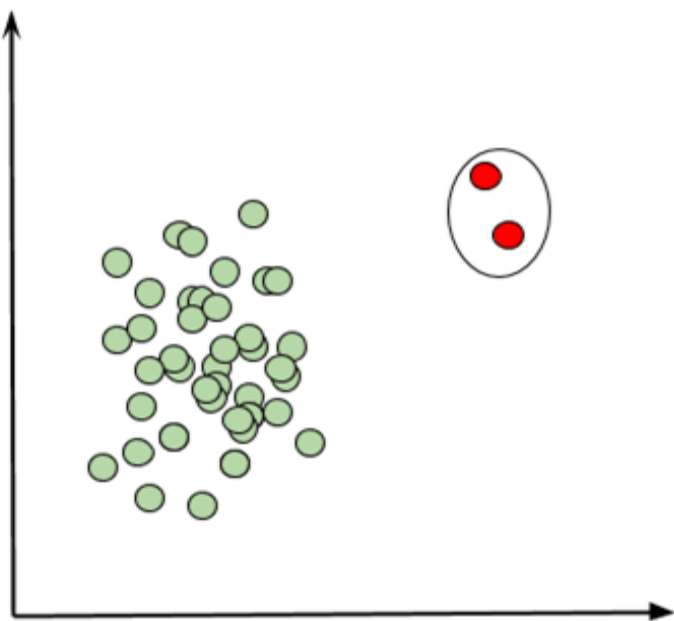
- 基因表达数据聚类



来源: Genome Biology 2007

聚类应用例子

- 异常值检测



来源: Floydhub

聚类任务

- 无监督学习
- 需要数据，但不需要标签
- 用于数据探索和模式发现：
 - 市场细分
 - 客户购物模式
 - 图像的区域
- 可独立使用，也可作为分类的前序步骤
- 当你不知道自己在寻找什么时很有用
- 缺点：有时也有引起错误的划分



From freecodecamp

监督学习（分类） vs 无监督学习（聚类）

- 分类 (**Classification**)

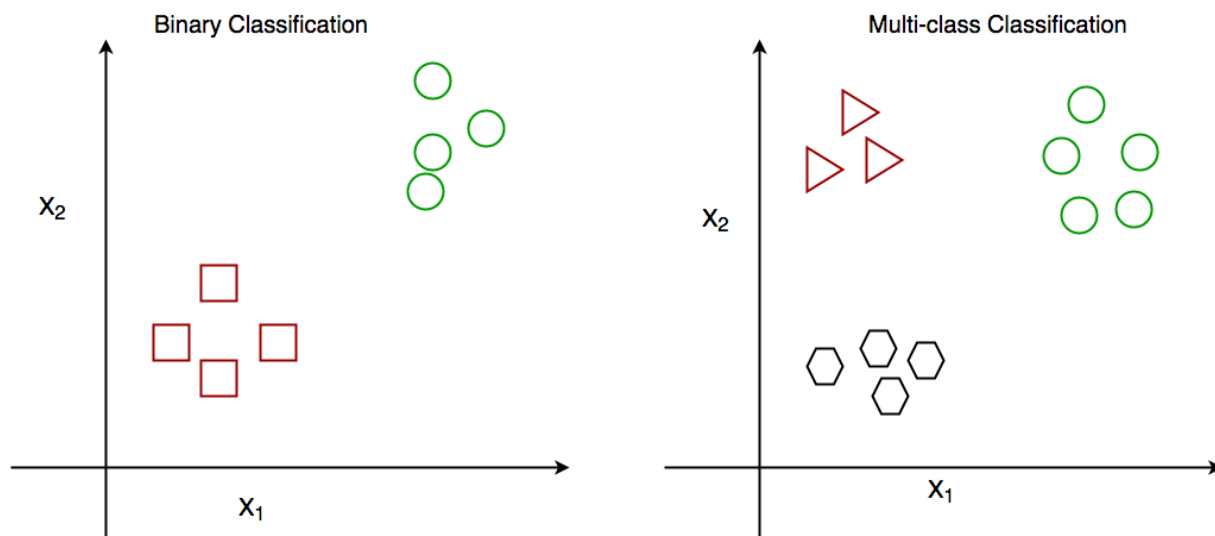
将数据根据预先定义的标签分配到不同类别的过程。

- 聚类 (**Clustering**)

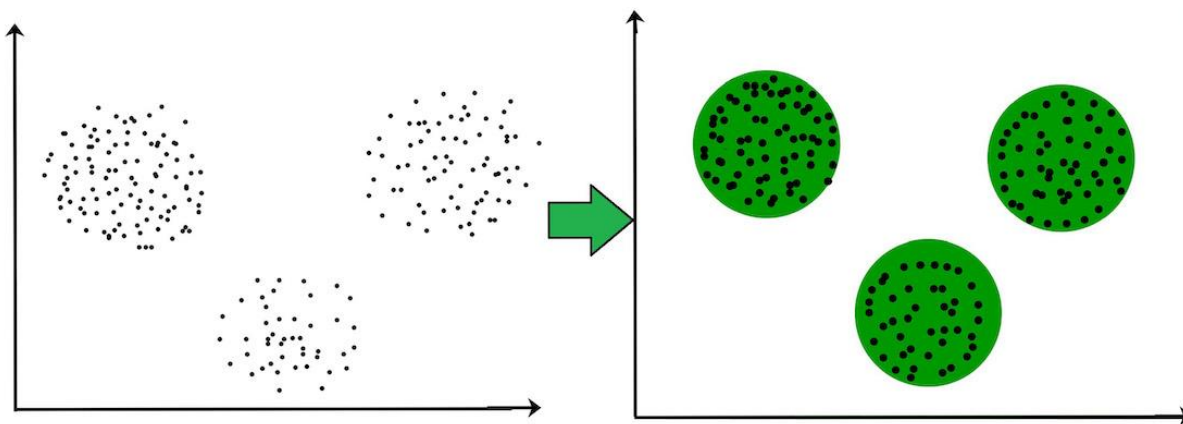
将一个未标记的数据集划分为相似对象的簇的过程。

监督学习（分类） vs 无监督学习（聚类）

分类



聚类



监督学习（分类） vs 无监督学习（聚类）

	分类（ Classification ）	聚类（ Clustering ）
目标	将实例分配到预定义的类别	根据相似性将相似的实例分组
目的	预测测试实例的类别或标签	发现数据中的固有模式或结构
训练	需要带标签的数据进行训练	不需要带标签的数据
输出	类别或标签分配	簇的分配
示例	预测电子邮件是否为垃圾邮件	根据购买行为对客户进行分组

大纲

- 聚类任务
- **性能度量**
- 距离计算
- K均值算法

性能度量

□ 性能度量(有效性指标)

是衡量聚类效果好坏的评判依据

● 核心思想

物以类聚：同一簇的样本尽可能彼此相似，不同簇的样本尽可能不同

聚类结果：簇内相似度高，簇间相似度低

- 外部指标：将聚类结果与某个“参考模型”进行比较
- 内部指标：直接考察聚类结果而不用任何参考模型

性能度量

- 给定数据集 $D = \{x_1, x_2, \dots, x_m\}$
- 聚类划分簇为 $C = \{C_1, C_2, \dots, C_k\}$
- 参考模型给出的簇的划分为 $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$

令 λ 与 λ^* 分别表示 C 与 C^* 对应的簇标记向量，将样本两两配对

在 C 与 C^* 中均隶属于相同簇的样本对数

$$a = |SS|, SS = \{(x_i, x_j) | \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$b = |SD|, SD = \{(x_i, x_j) | \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

$$c = |DS|, DS = \{(x_i, x_j) | \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$d = |DD|, DD = \{(x_i, x_j) | \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

在 C 中隶属于同一簇，
 C^* 中均隶属于不同簇
的样本对数

$$a + b + c + d = \frac{m(m-1)}{2}$$

在 C 与 C^* 中均隶属于
不同簇的样本对数

性能度量

□ 外部指标

- Jaccard系数 (JC)

$$JC = \frac{a}{a+b+c}$$

- FM指数 (FMI)

$$FMI = \sqrt{\frac{a}{a+b} \cdot \frac{a}{a+c}}$$

- Rand指数 (RI)

$$RI = \frac{2(a+b)}{m(m-1)}$$

[0,1]区间内,
越大越好.

性能度量

□ 内部指标

考虑聚类结果的簇划分 $C = \{C_1, C_2, \dots, C_k\}$

- 簇 C 内样本间的平均距离

$$avg(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$

- 簇 C 内样本间的最远距离

$$diam(C) = \max_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$

- 簇 C_i 与簇 C_j 最近样本间的距离

$$d_{min}(C) = \min_{x_i \in C_i, x_j \in C_j} dist(x_i, x_j)$$

- 簇 C_i 与簇 C_j 中心点间的距离

$$d_{cen}(C) = dist(\mu_i, \mu_j)$$

性能度量 - 内部指标

➤ DB指数 (DBI)

$$DBI = \frac{1}{k} \sum_{i=1}^k \text{Max}_{j \neq i} \left(\frac{\text{avg}(C_i) + \text{avg}(C_j)}{d_{cen}(\mu_i, \mu_j)} \right)$$

越小越好

➤ Dunn指数 (DI)

$$DI = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{min}(C_i, C_j)}{\max_{1 \leq l \leq k} \text{diam}(C_l)} \right) \right\}$$

越大越好

大纲

- 聚类任务
- 性能度量
- 距离计算
- K均值算法

距离计算

- 距离度量的性质

非负性: $dist(x_i, x_j) \geq 0$

同一性: $dist(x_i, x_j) = 0$ 当且仅当 $x_i = x_j$

对称性: $dist(x_i, x_j) = dist(x_j, x_i)$

直递性: $dist(x_i, x_j) \leq dist(x_i, x_k) + dist(x_k, x_j)$

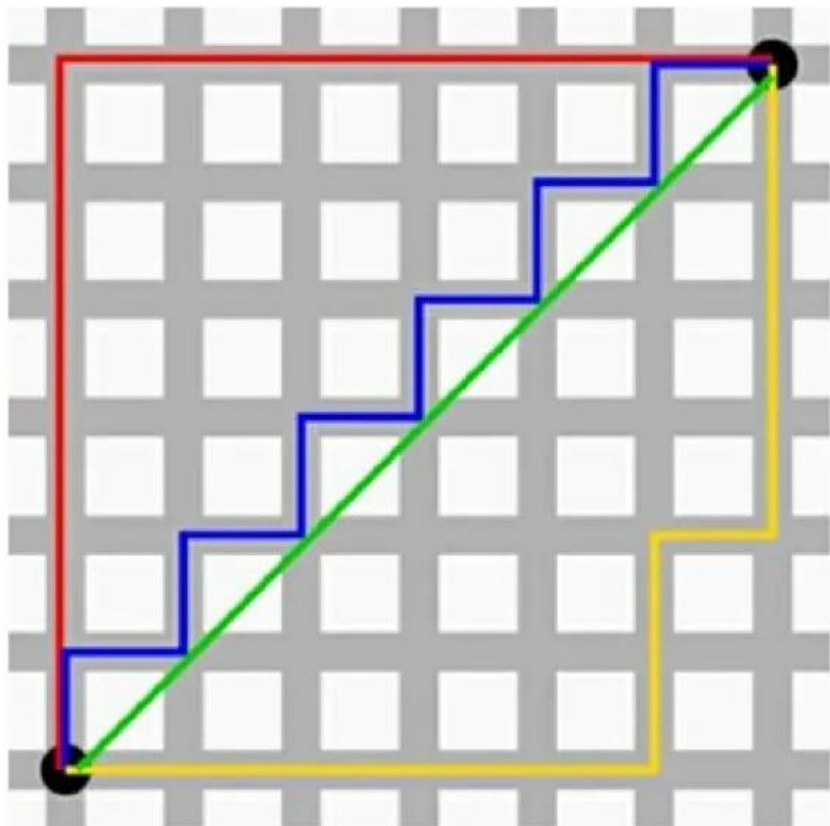
- 常用距离

闵可夫斯基距离:

$$dist(x_i, x_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{\frac{1}{p}}$$

- $p = 2$: 欧氏距离
- $p = 1$: 曼哈顿距离 (亦称“街区距离”)

距离计算



- 绿线：欧氏距离
(Euclidean distance)
 - 红线：曼哈顿距离
(Manhattan distance)
- (注：蓝线、黄线和红线距离一样)

距离计算

□ 属性介绍

➤ 连续属性

在定义域上有**无穷多**个可能的取值（其取值一般直接有“序的关系”）

➤ 离散属性

在定义域上是**有限个**可能的取值

➤ 有序属性

可以直接**在属性上计算距离**：定义域为{1,2,3}的离散属性，“1”与“2”比较接近、与“3”比较远，这是有序属性

➤ 无序属性

无法在属性上直接进行**距离计算**：定义域为{飞机，火车，轮船}这样的离散属性

距离度量

□ VDM距离

用来处理无序属性，属性 u 上取离散值 a 与 b 之间的VDM距离为

$m_{u,a,i}$: 第 i 个样本簇中在属性 u 上取值为 a 的样本数

k : 样本簇数

$$VDM_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p$$

$m_{u,a}$: 属性 u 上取值为 a 的样本数

距离度量

□ VDM距离

西瓜数据集3.0

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	0.774	0.376	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	0.634	0.264	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	0.608	0.318	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	0.556	0.215	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	0.403	0.237	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	0.481	0.149	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	0.437	0.211	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	0.666	0.091	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	0.243	0.267	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	0.245	0.057	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	0.343	0.099	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	0.639	0.161	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	0.657	0.198	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	0.360	0.370	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	0.593	0.042	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	0.719	0.103	否

假设划分为好/坏两簇： $m_{u=\text{色泽},a=\text{青绿}} = 6$, $m_{u=\text{色泽},a=\text{青绿},i=\text{好瓜}} = 3$

距离度量

□ MinkovDM_p距离 (闵可夫斯基距离和VDM距离结合)

用来处理混和属性

n_c : 有序属性个数

$n - n_c$: 无序属性个数

$$MinkovDM_p(x_i, x_j) = \left(\sum_{u=1}^{n_c} |x_{iu} - x_{ju}|^p + \sum_{u=n_c+1}^n VDM_p(x_{iu}, x_{ju}) \right)^{\frac{1}{p}}$$

● 加权距离

当样本空间中不同属性的重要性不同时, 通常给每个属性增加权重

$$dist(x_i, x_j) = (\omega_1 \cdot |x_{i1} - x_{j1}|^p + \cdots + \omega_n \cdot |x_{in} - x_{jn}|^p)^{\frac{1}{p}}$$

ω_i : 不同属性的权重值, $\sum_i^n \omega_i = 1$

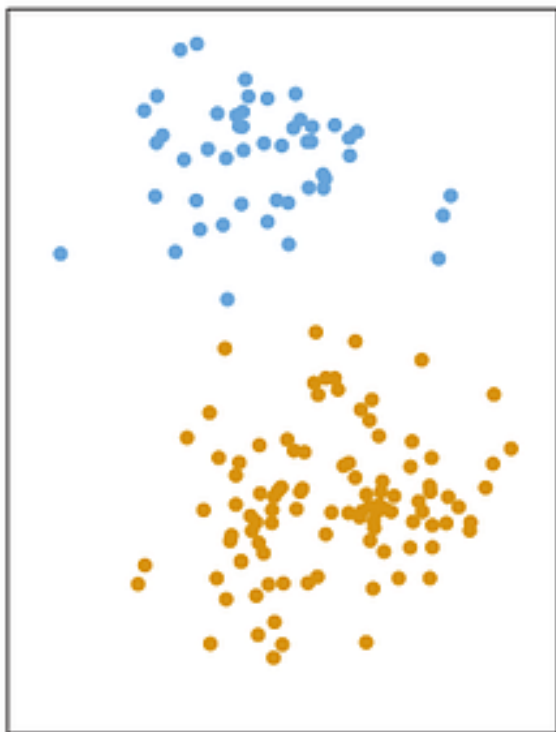
大纲

- 聚类任务
- 性能度量
- 距离计算
- K均值算法

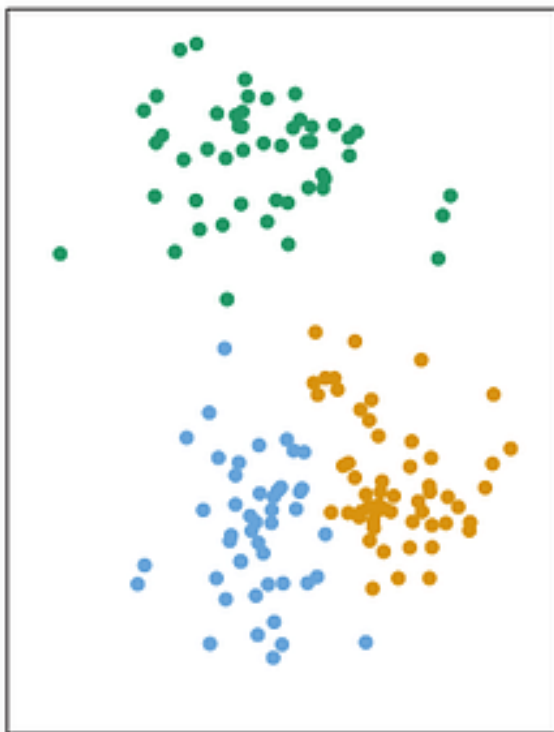
原型聚类 - k 均值算法

一种将数据集划分为 K 个不同部分的简单方法

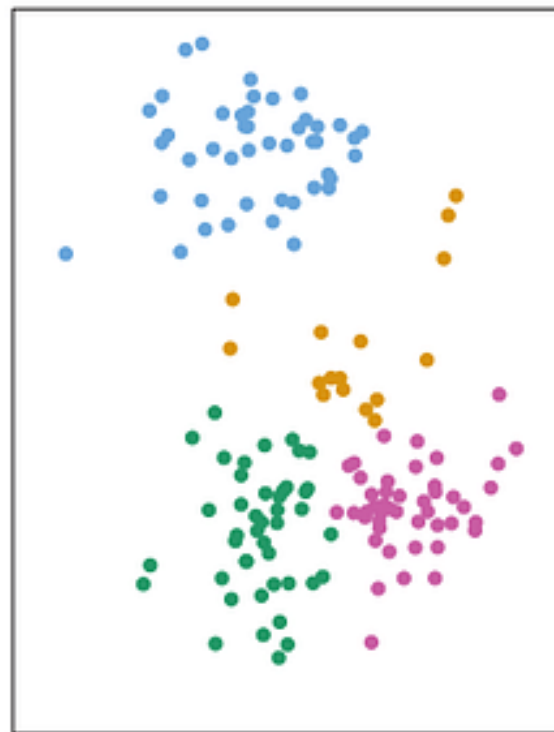
K=2



K=3



K=4



原型聚类 - k 均值算法

□ k 均值算法

给定包含 m 个无标记样本的样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, 聚类划分簇为 $C = \{C_1, C_2, \dots, C_k\}$

➤ k 均值目标: 最小化聚类划分簇的平方误差 E

$\mu_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$: 簇 C_i 的均值向量

$$E = \sum_{i=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mu_i\|_2^2$$

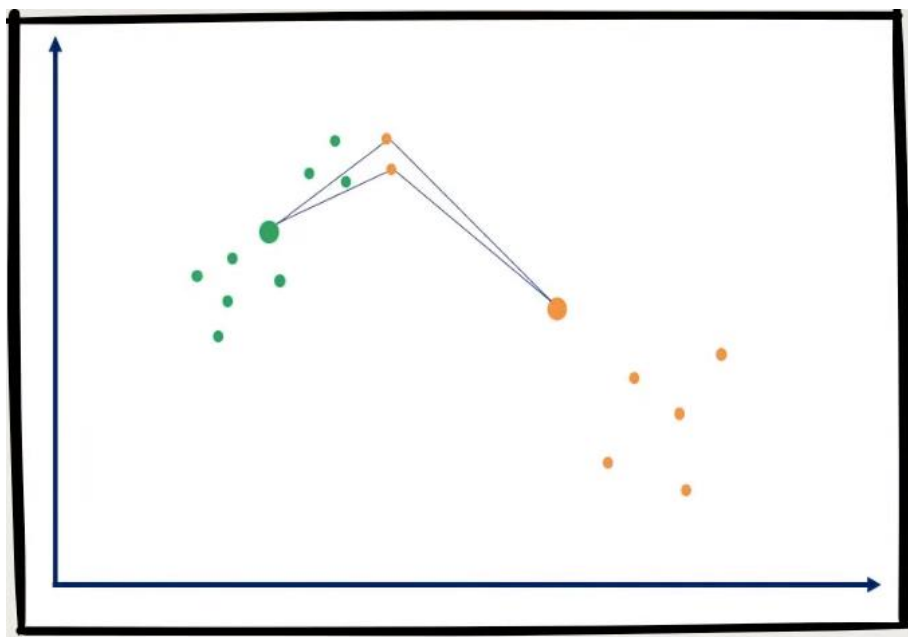

E 在一定程度上刻画了簇内样本围绕簇均值向量的紧密程度:
 E 越小, 簇内样本相似度越高

性能度量

● 核心思想

物以类聚：同一簇的样本尽可能彼此相似，不同簇的样本尽可能不同

聚类结果：簇内相似度高，簇间相似度低



(1) 最小化同一簇内点之间的距离，即类内平方和 (WSS, Within-cluster Sum of Squares)

(2) 最大化簇间距离

原型聚类 - k 均值算法

□ 算法流程

初始化每个簇的均值向量

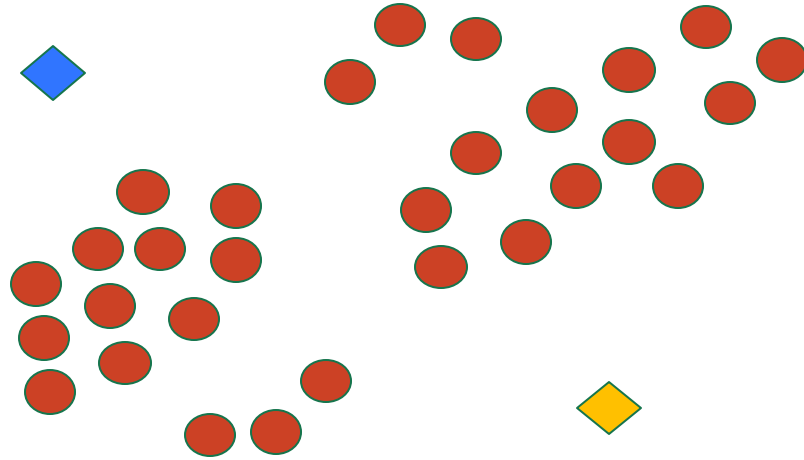
repeat

1. (更新) 簇划分;
2. 计算每个簇的均值向量

until 当前均值向量均不需要更新

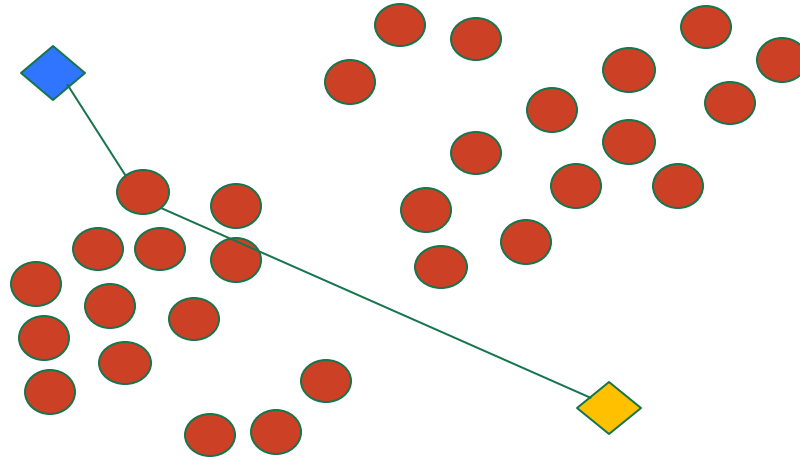
k 均值算法：例子1

- 初始化 2 个随机点作为聚类中心



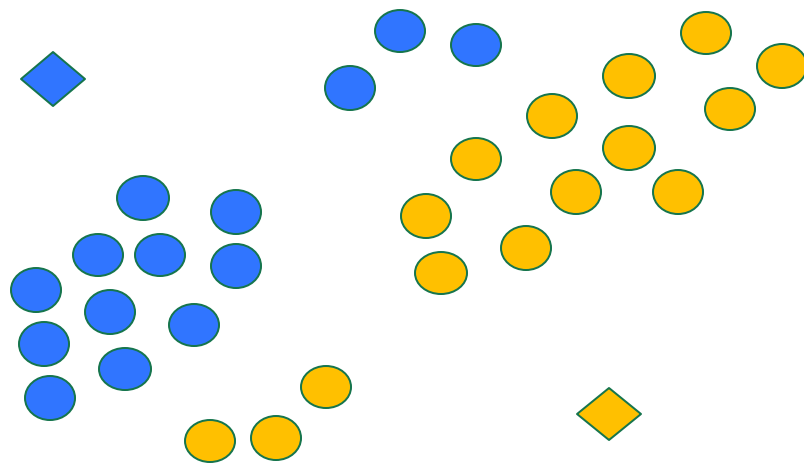
k 均值算法：例子1

- 计算到每个聚类中心的距离（相似性度量）



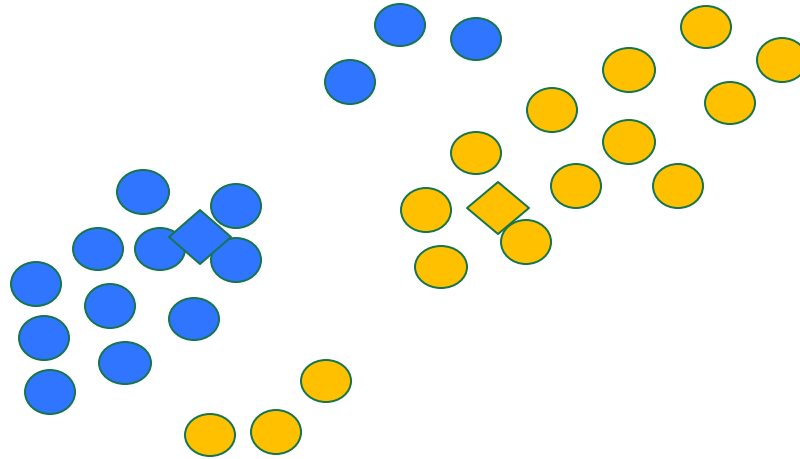
k 均值算法：例子1

- 迭代一：将数据点分配到最近的聚类中心



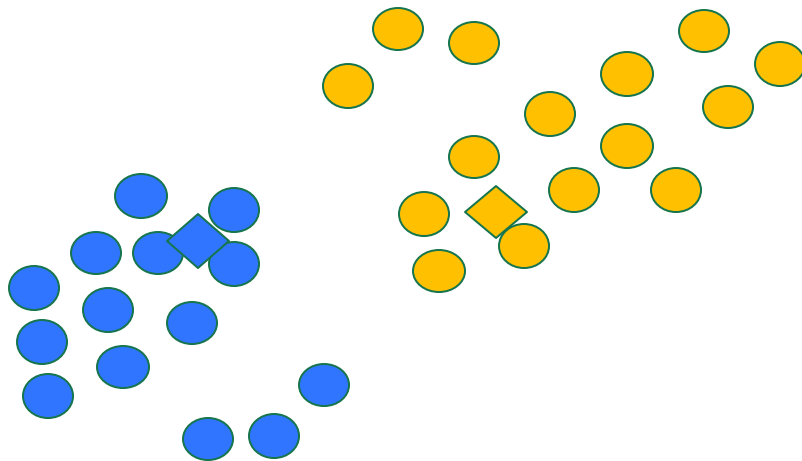
k 均值算法：例子1

- 迭代一：更新聚类中心



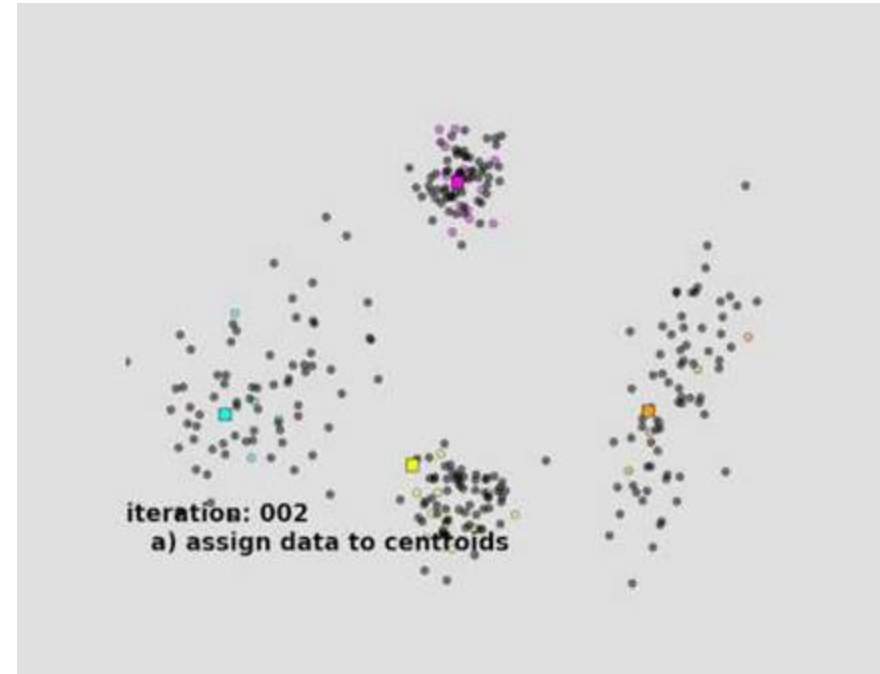
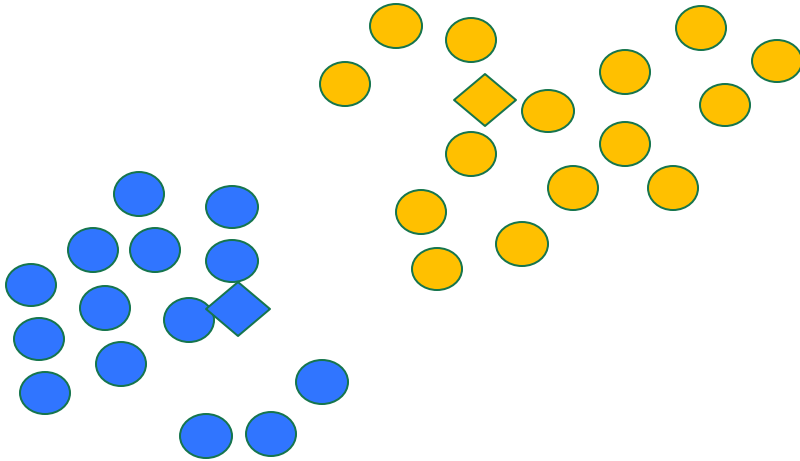
k 均值算法：例子1

- 迭代二：将数据点分配到最近的聚类中心



k 均值算法：例子1

- 重复迭代直至收敛



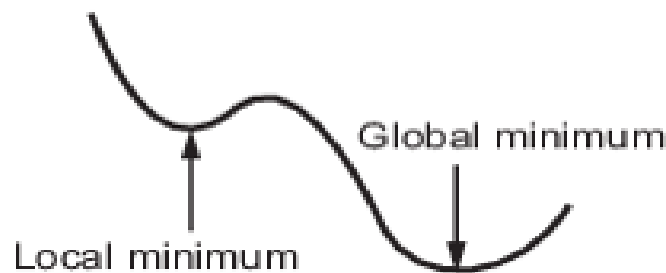
k 均值算法

□ 停止准则

- 如何定义收敛？
 - 没有数据点改变聚类
 - 距离总和最小化
 - 达到最大迭代次数
- 该算法保证收敛到一个结果
(但可能是局部最优)



这是该如何处理呢？

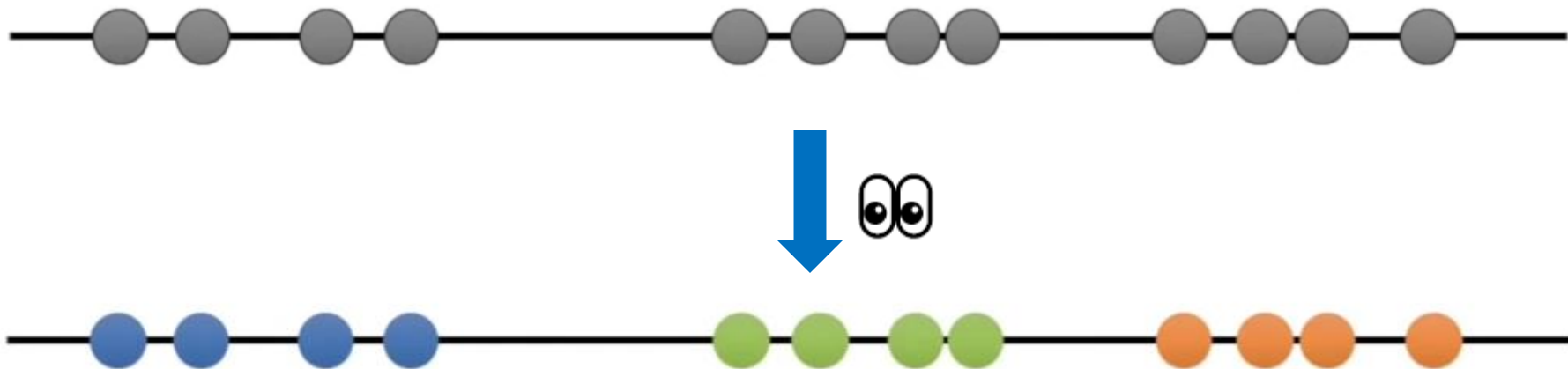


From: Mathworks

k 均值算法：例子2

对于这些数据，你知道需要将其分成 3 个簇。

这可能是来自 3 种不同类型的肿瘤或其他细胞类型的测量数据。



k 均值算法：例子2

步骤1： 选择要识别的聚类数量，即确定 K 的取值。

(这就是“ K 均值聚类”中的“ K ”)

假设 $K=3$



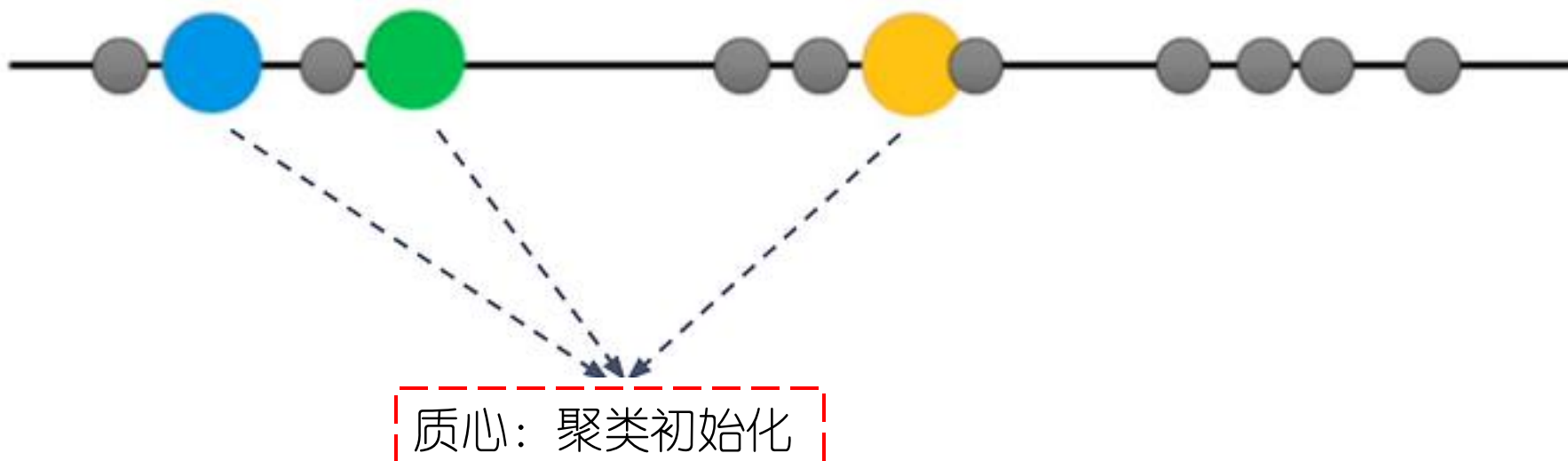
问题：如何确定 K 的值呢？

(本章后面会有介绍)

k 均值算法：例子2

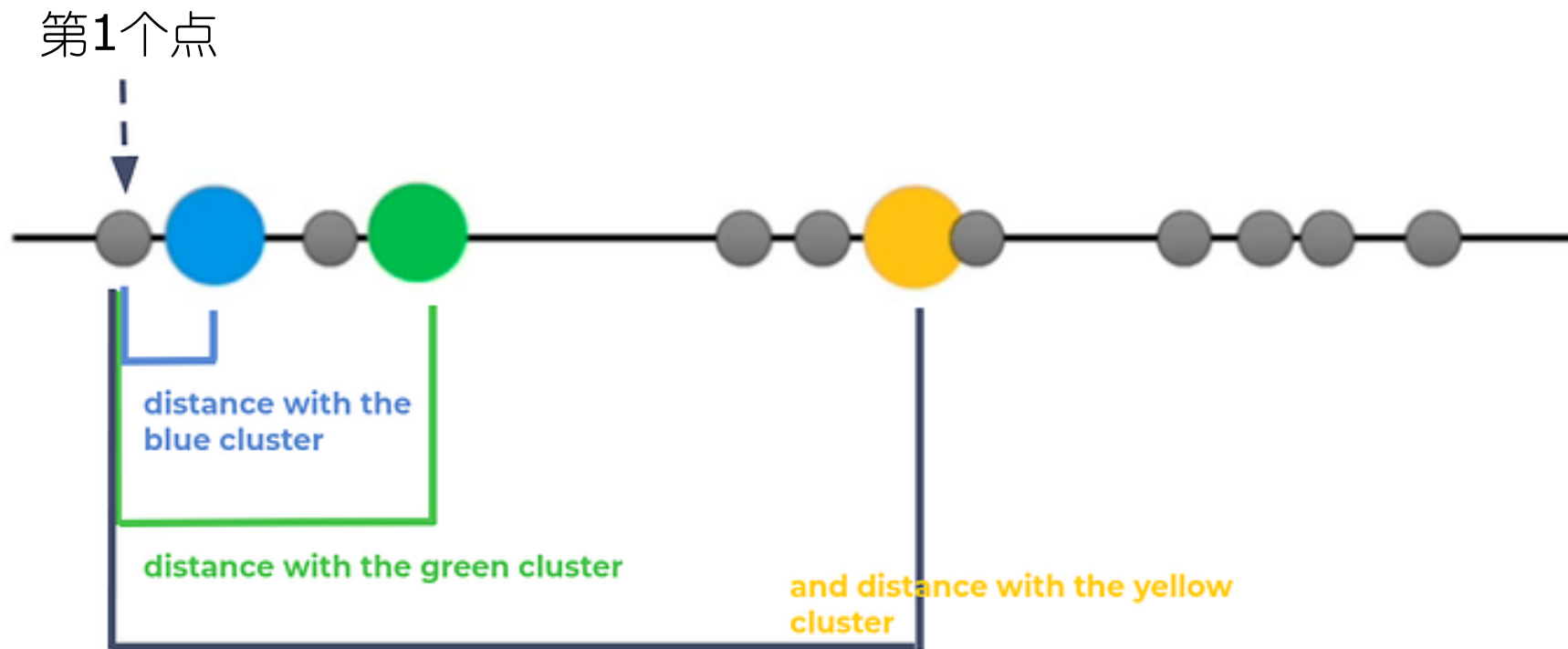
步骤2：随机选择 3 个不同的质心。

（新数据点作为聚类初始化）



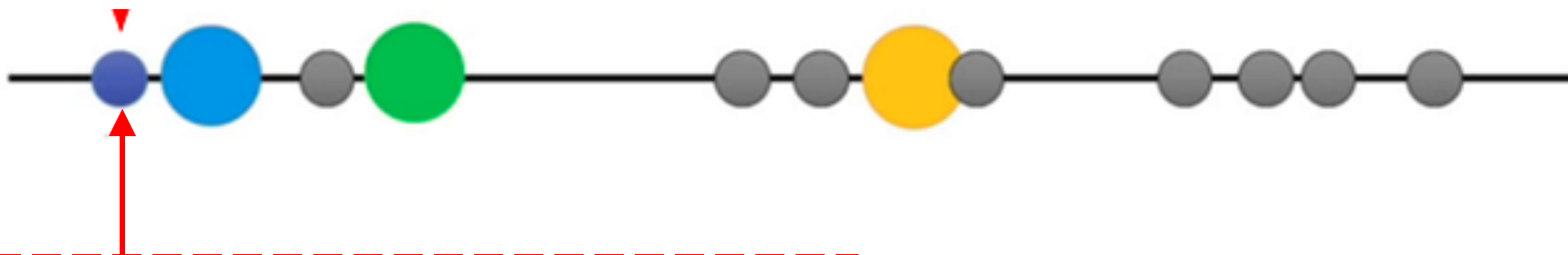
k 均值算法：例子2

步骤3：测量每个点与质心之间的距离。



k 均值算法：例子2

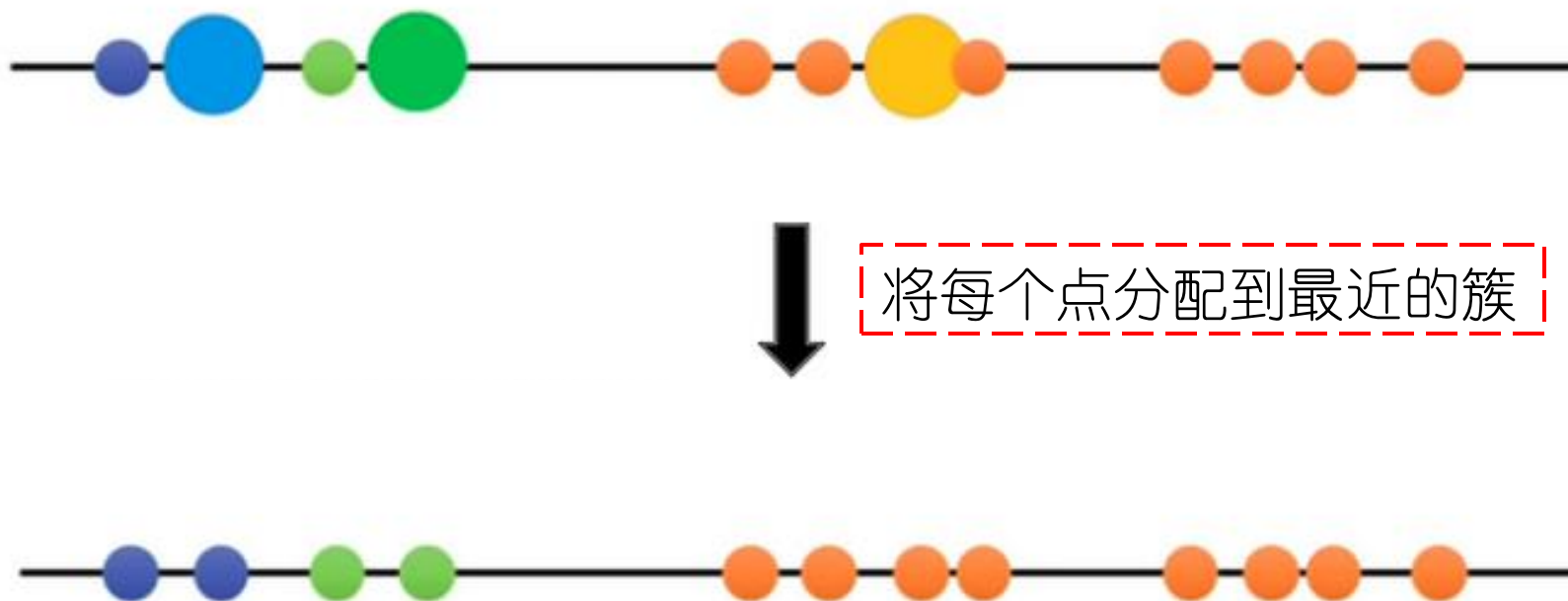
步骤3：测量每个点与质心之间的距离。



由于第1个点离蓝色质心最近，因此把这个点归为蓝色质心这一簇。

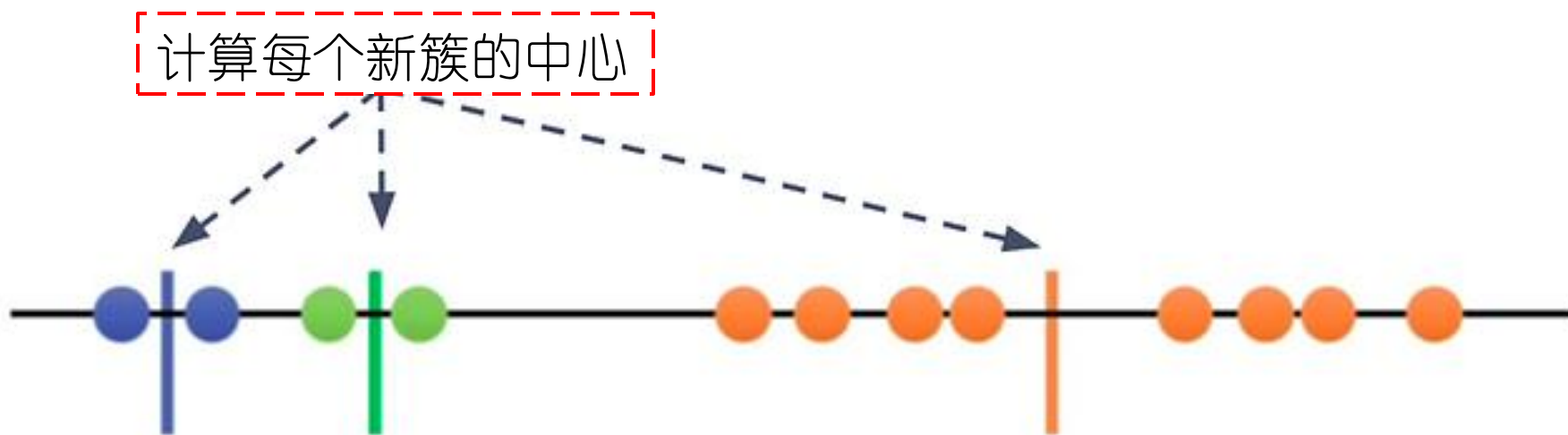
k 均值算法：例子2

步骤4：分别对其他点进行步骤3的处理，将每个点分配到最近的簇。



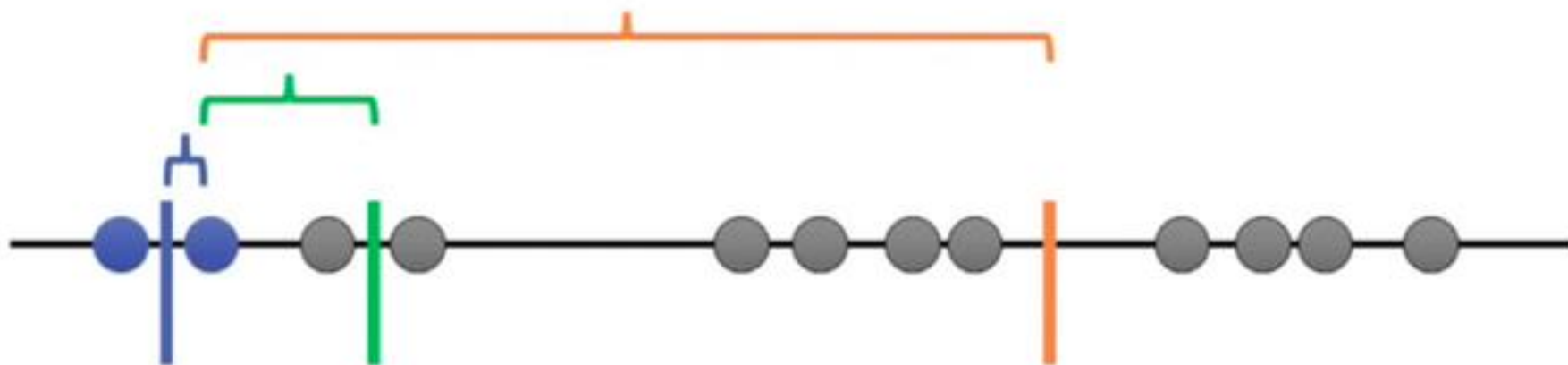
k 均值算法：例子2

步骤5：计算每个新簇的中心，并更新为新的质心。



k 均值算法：例子2

步骤6：使用更新后的新质心，重复步骤3-5的计算。



k 均值算法：例子2

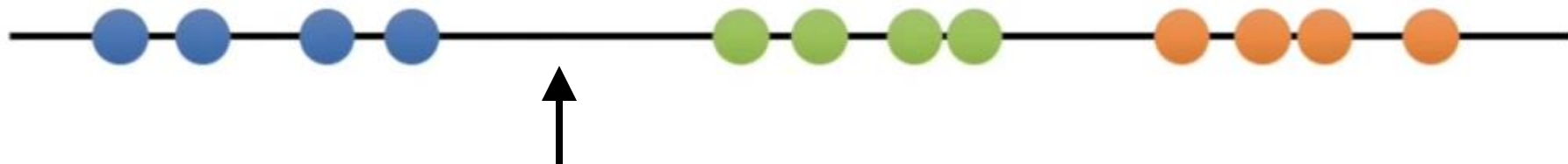
- 重复迭代计算，直到收敛才停止
 - 没有数据点改变聚类
 - 达到最大迭代次数



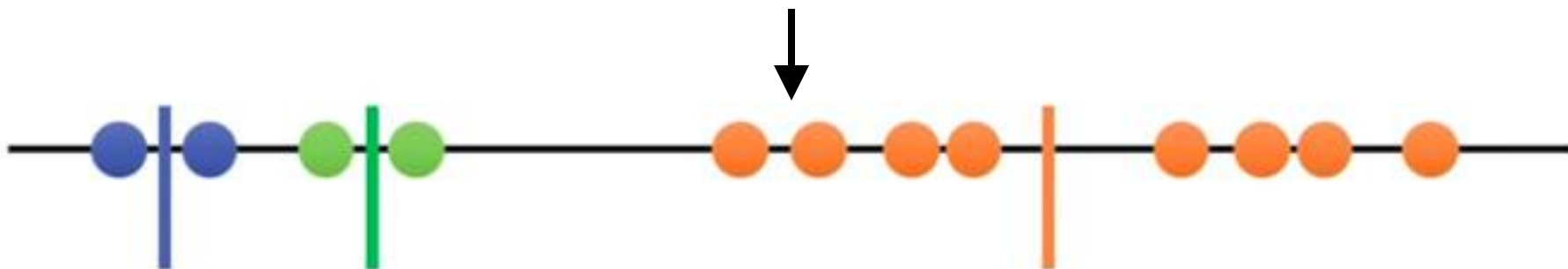
k 均值算法：例子2



k 均值算法：例子2



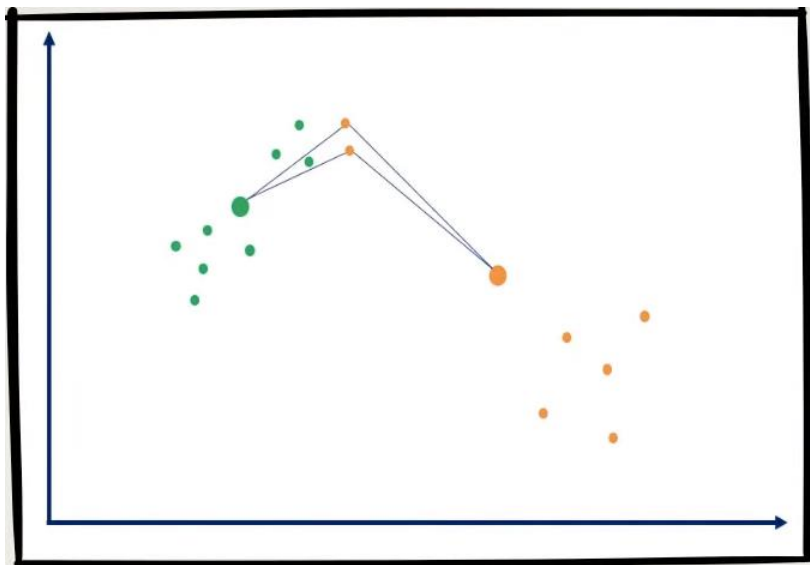
与我们用眼睛观察的结果相比，
K 均值聚类の結果看上去很糟糕。



k 均值算法：例子2

- 这个过程完成了吗？当然没有。

K 均值算法旨在选择能够最小化类内平方和（WSS）的质心。



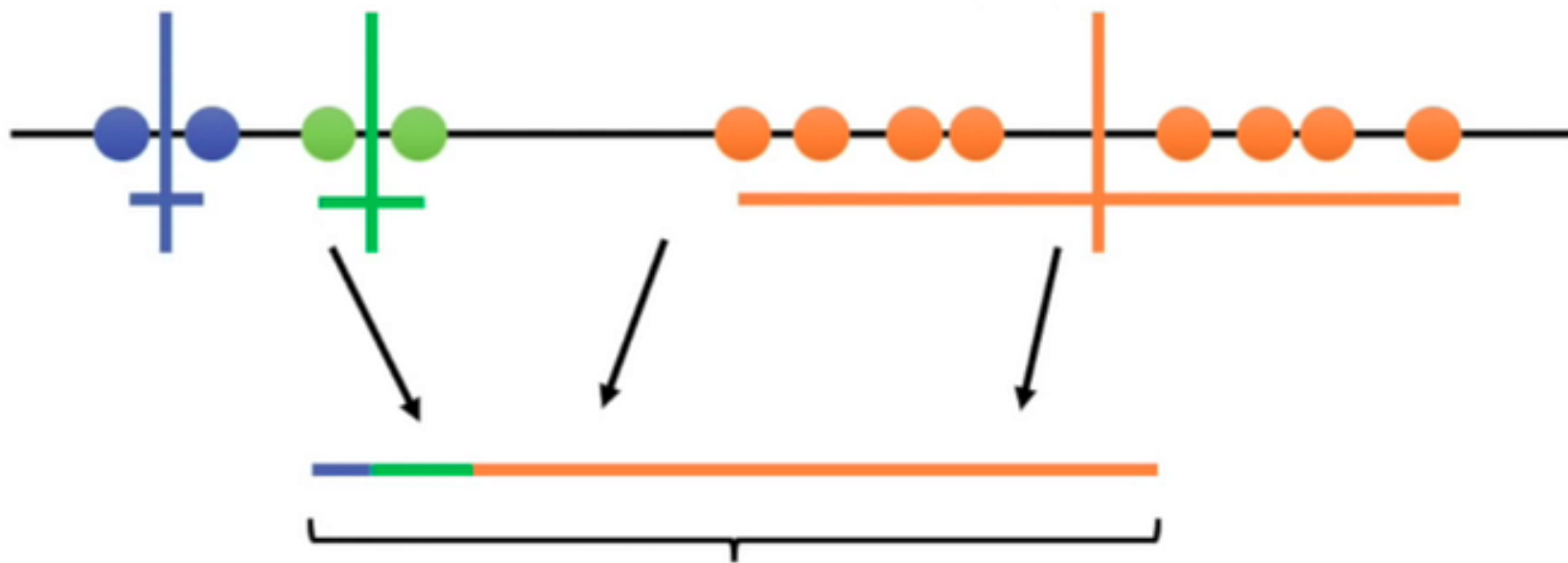
(1) 最小化同一簇内点之间的距离，即类内平方和误差
(WSS, Within-cluster Sum of Squares error)

(2) 最大化簇间距离

- 如何评估这个聚类的结果？？

k 均值算法：例子2

步骤7：计算每一簇内的点与对应质心的离散程度。



类内总变异 (Total variation within the clusters)

通过将所有簇内的每个点与对应质心的距离相加，我们可以得到一个整体的评估指标：类内总变异 (Total variation within the clusters)。

k 均值算法：例子2

回到一开始的步骤**1**，重新进行第**二**次尝试



k 均值算法：例子2

重复步骤2-7，直到类内总变异最小。

步骤2



步骤3-4



步骤5



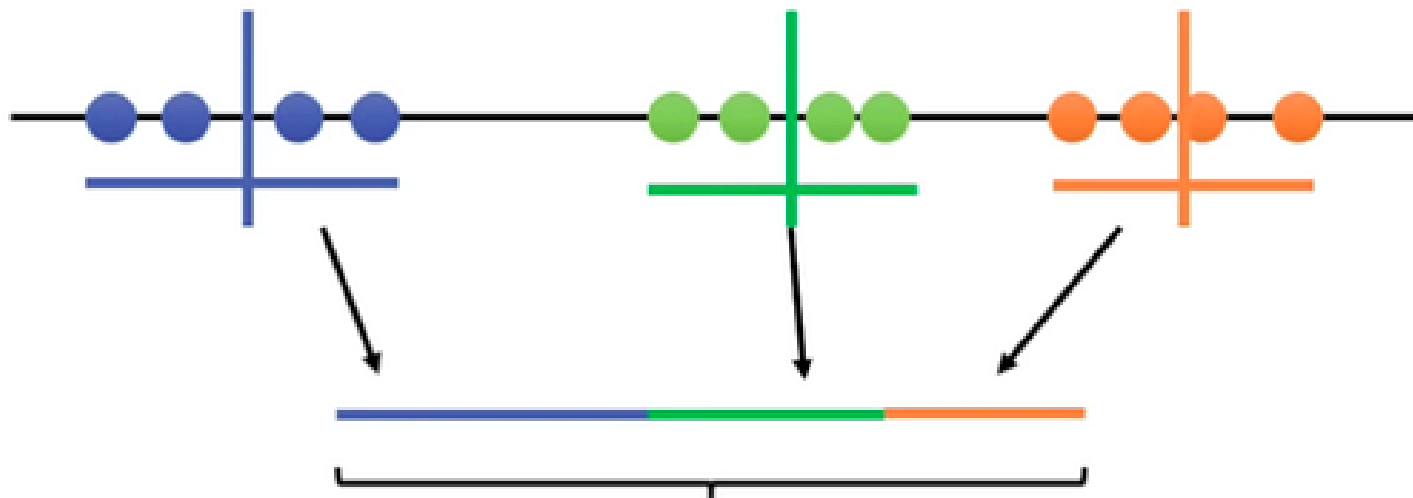
步骤6



重复迭代计算
直到收敛

k 均值算法：例子2

步骤7



类内总变异 (Total variation within the clusters)

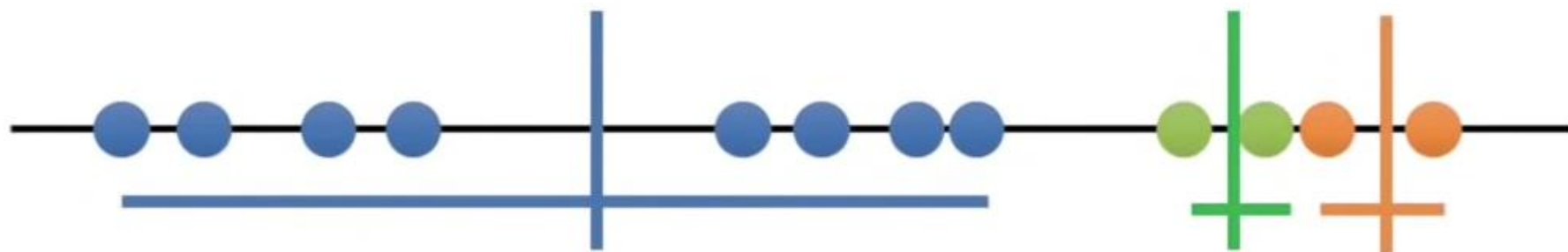
k 均值算法：例子2

回到一开始的步骤**1**，重新进行第**三**次尝试



k 均值算法：例子2

重复步骤2-7，直到类内总变异最小。



k 均值算法：例子2

第一次尝试



第二次尝试



最优的尝试

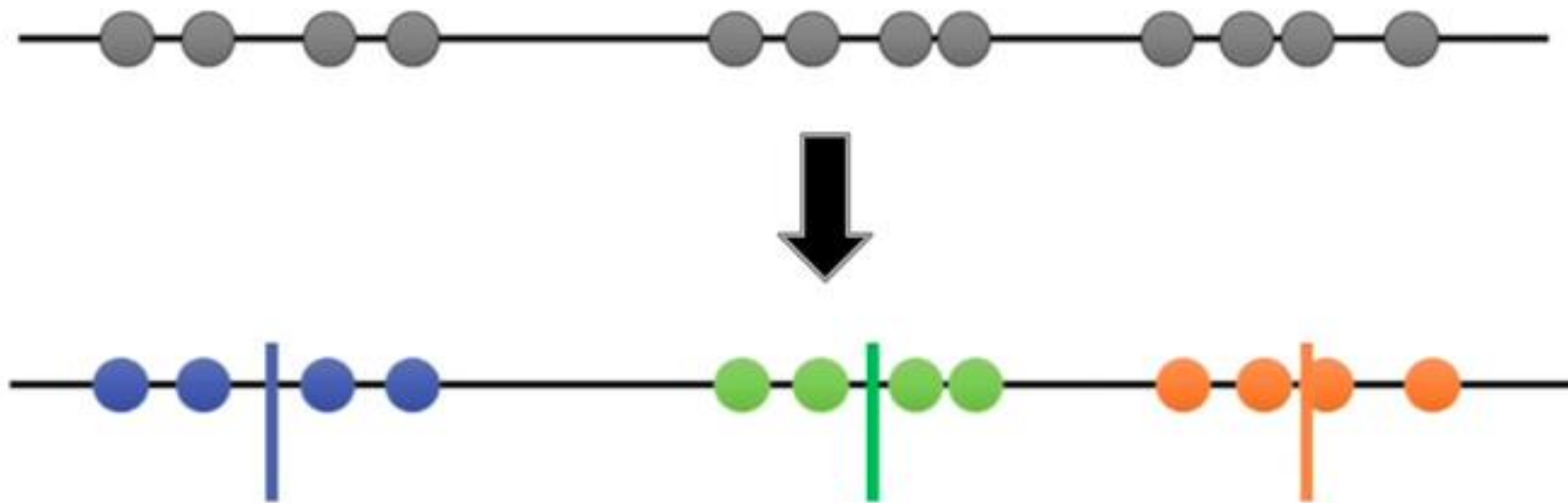
第三次尝试



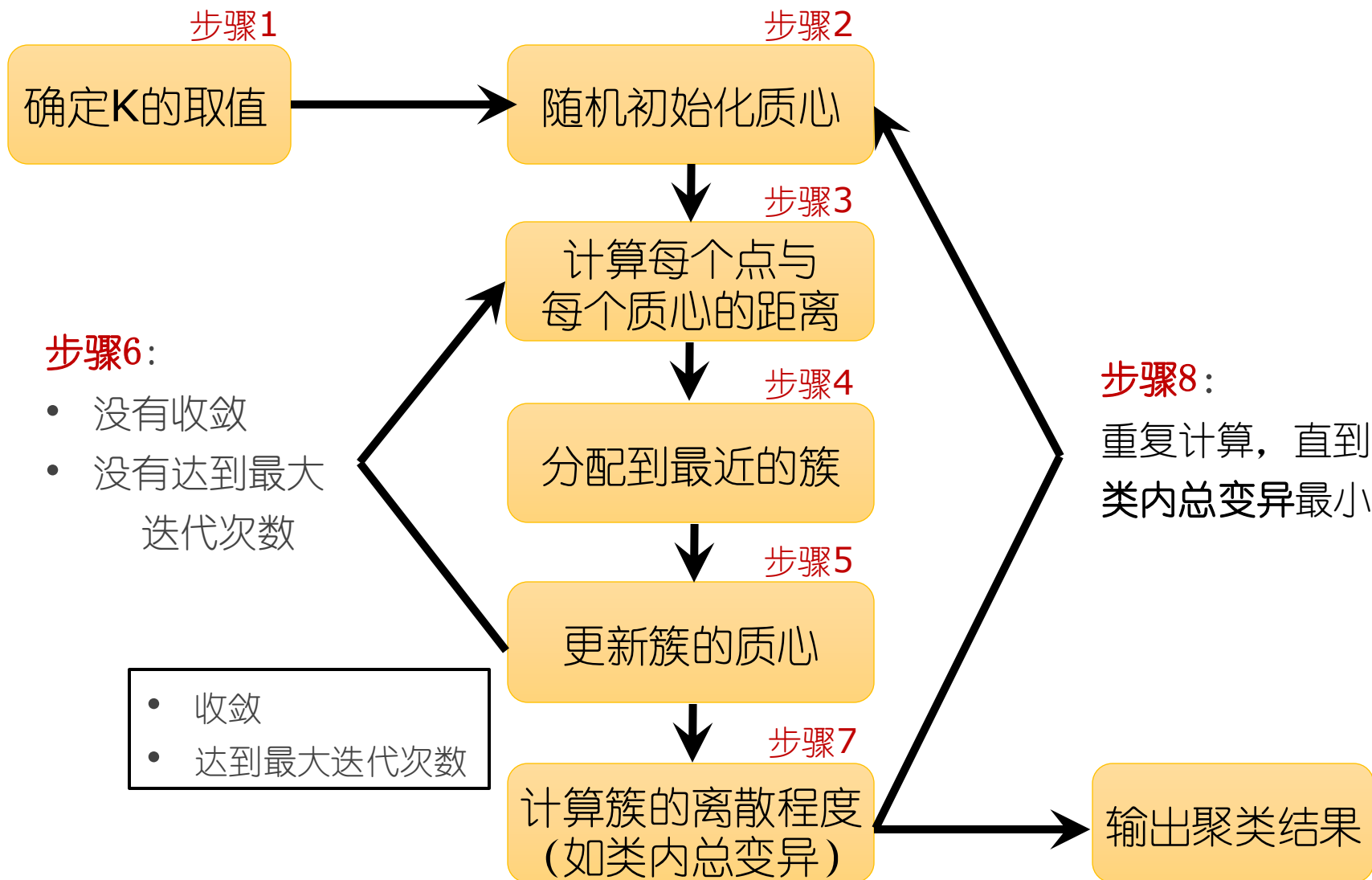
通过对比类内总变异，第二次尝试的结果最优

k 均值算法：例子2

此次聚类的最终结果如下：



k均值算法：小结



k 均值算法：计算例子

问题描述：

假设您有以下五个二维点集：

$\{(2, 3), (3, 3), (8, 3), (12, 3), (10, 4)\}$ 。

请使用 K 均值算法进行聚类，其中 $K=2$ 。

选择的初始质心为：

质心1: $(2, 3)$

质心2: $(8, 3)$

k 均值算法：计算例子

任务说明：

1. 使用**欧氏距离**公式计算每个点与两个质心之间的距离。
2. 确定每个点所属的聚类。
3. 计算每个聚类的新质心。
4. 重复上述过程，直到质心不再变化。

原型聚类 - k 均值算法

□ 算法伪代码

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
聚类簇数 k .

过程:

```
1: 从  $D$  中随机选择  $k$  个样本作为初始均值向量  $\{\mu_1, \mu_2, \dots, \mu_k\}$ 
2: repeat
3:   令  $C_i = \emptyset$  ( $1 \leq i \leq k$ )
4:   for  $j = 1, \dots, m$  do
5:     计算样本  $\mathbf{x}_j$  与各均值向量  $\mu_i$  ( $1 \leq i \leq k$ ) 的距离:  $d_{ji} = \|\mathbf{x}_j - \mu_i\|_2$ ;
6:     根据距离最近的均值向量确定  $\mathbf{x}_j$  的簇标记:  $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$ ;
7:     将样本  $\mathbf{x}_j$  划入相应的簇:  $C_{\lambda_j} = C_{\lambda_j} \cup \{\mathbf{x}_j\}$ ;
8:   end for
9:   for  $i = 1, \dots, k$  do
10:    计算新均值向量:  $\mu'_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$ ;
11:    if  $\mu'_i \neq \mu_i$  then
12:      将当前均值向量  $\mu_i$  更新为  $\mu'_i$ 
13:    else
14:      保持当前均值向量不变
15:    end if
16:  end for
17: until 当前均值向量均未更新
18: return 簇划分结果
```

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

原型聚类 - k 均值算法

□ 实例

西瓜数据集 4.0

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

原型聚类 - k 均值算法

□ k 均值算法实例

假定聚类簇数 $k=3$ ，算法开始时，随机选择3个样本 x_6, x_{12}, x_{27} 作为初始均值向量，即 $\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.533; 0.472)$ 。

考察样本 $x_1 = (0.697; 0.460)$ ，它与当前均值向量 μ_1, μ_2, μ_3 的距离分别为0.369, 0.506, 0.166，因此 x_1 将被划入簇 C_3 中。类似的，对数据集的所有样本考察一遍后，可得当前簇划分为

$$C_1 = \{x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{15}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$

$$C_3 = \{x_1, x_2, x_3, x_4, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

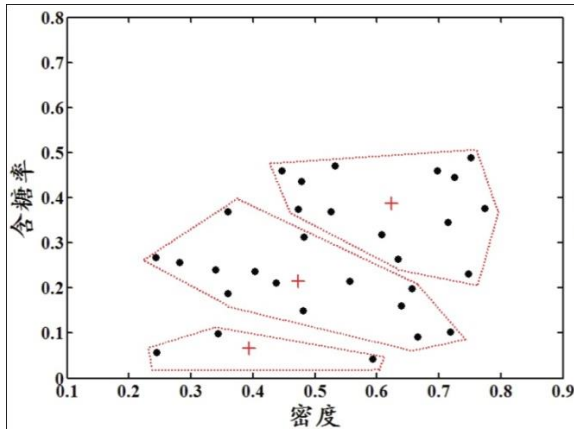
于是，可以从分别求得新的均值向量

$$\mu'_1 = (0.473; 0.214), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.623; 0.388)$$

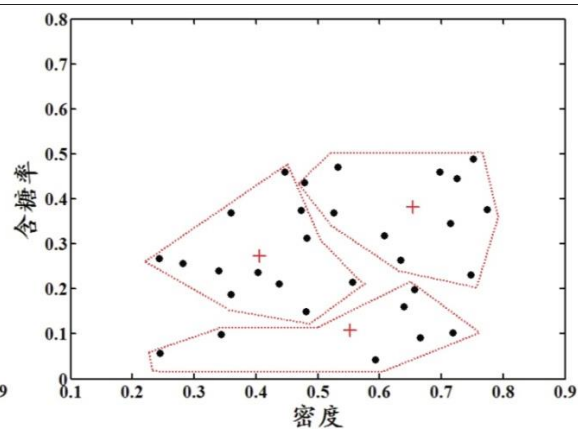
不断重复上述过程，如下图所示。

原型聚类 - k 均值算法

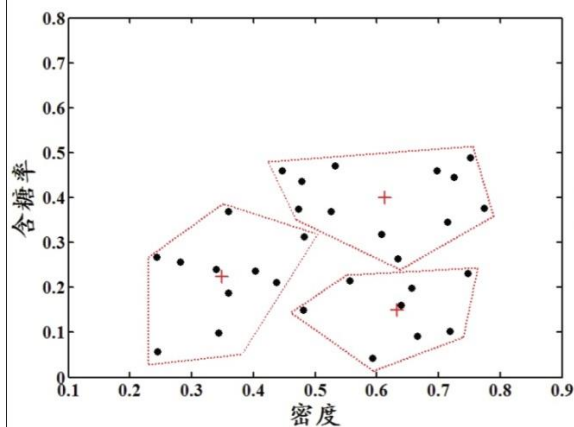
□ 聚类结果：



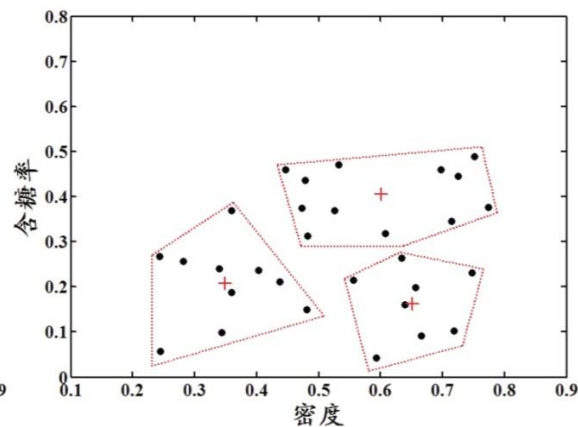
(a) 第一轮迭代后



(b) 第二轮迭代后



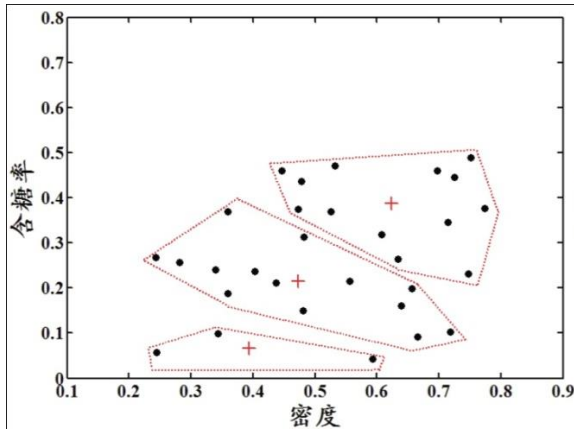
(c) 第三轮迭代后



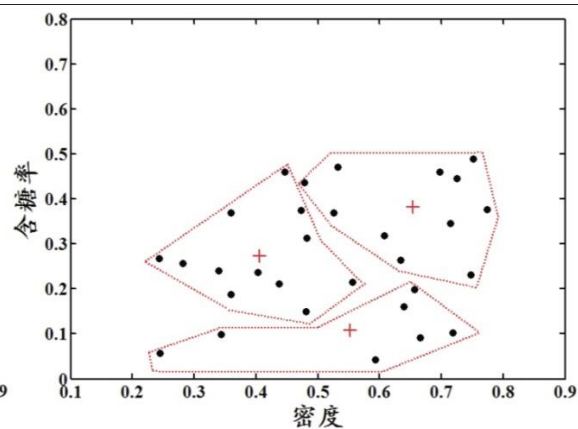
(d) 第四轮迭代后

原型聚类 - k 均值算法

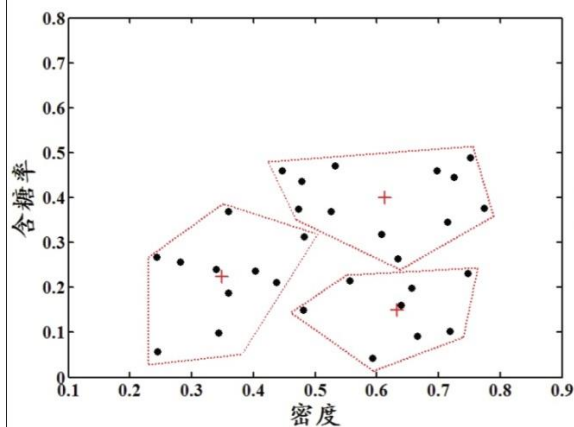
□ 聚类结果：



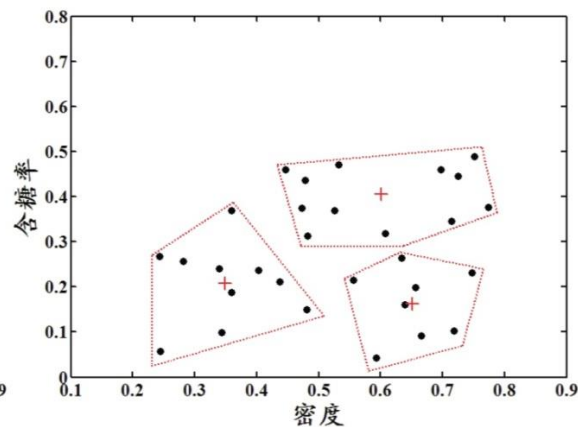
(a) 第一轮迭代后



(b) 第二轮迭代后



(c) 第三轮迭代后



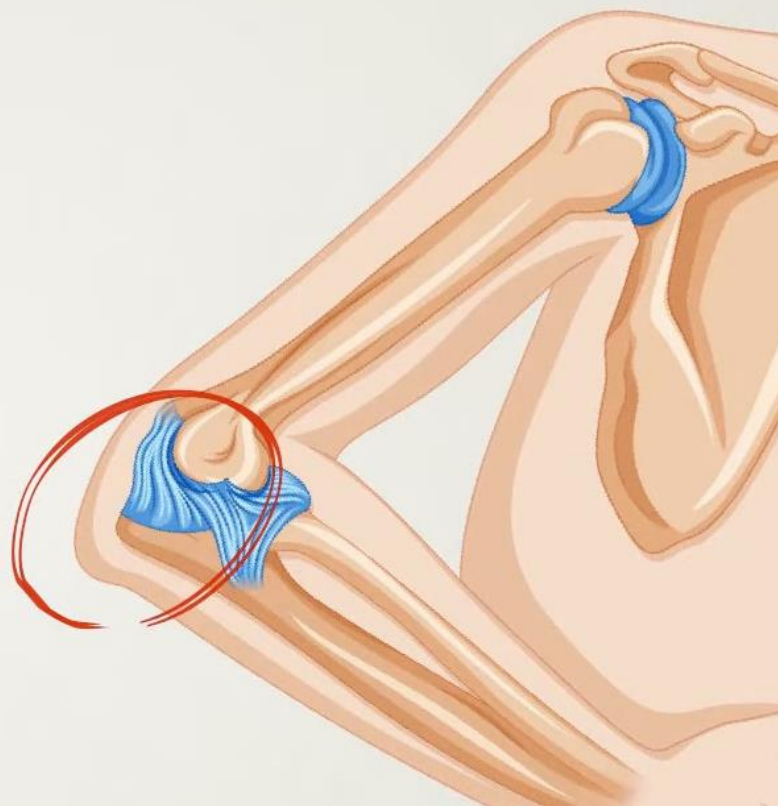
(d) 第四轮迭代后

k 均值算法：肘部法则

问题：如何确定K的值呢？

**THE ELBOW
METHOD**

（肘部法则）



k 均值算法：肘部法则

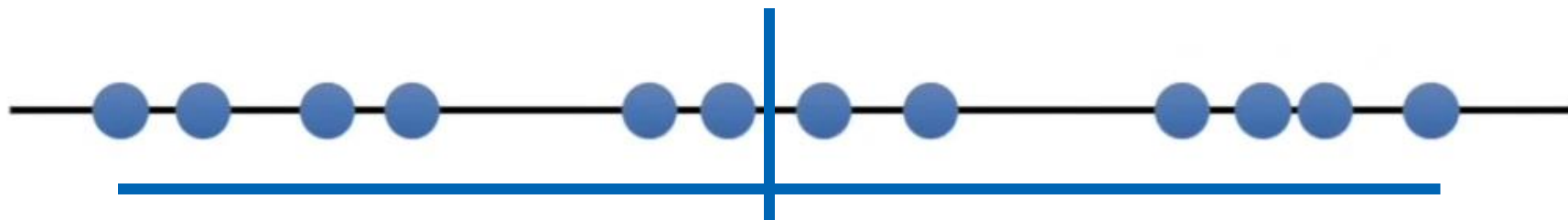
问题：如何确定K的值呢？



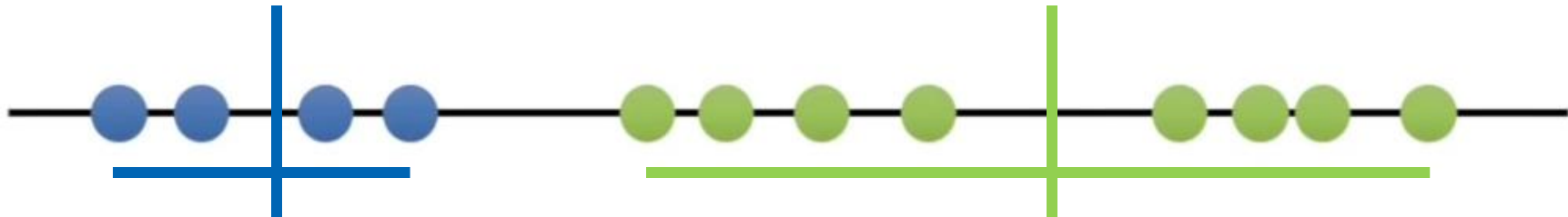
其中一个方法：尝试不同的 K 值。

k 均值算法：肘部法则

K=1



K=2



K=1

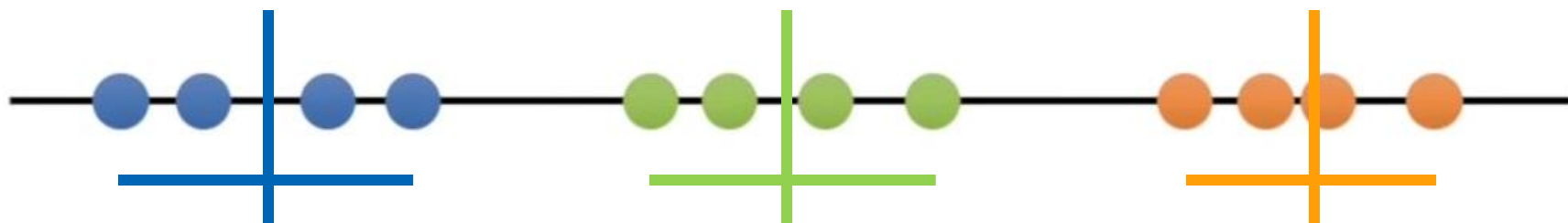


K=2



k 均值算法：肘部法则

K=3



K=1



K=2

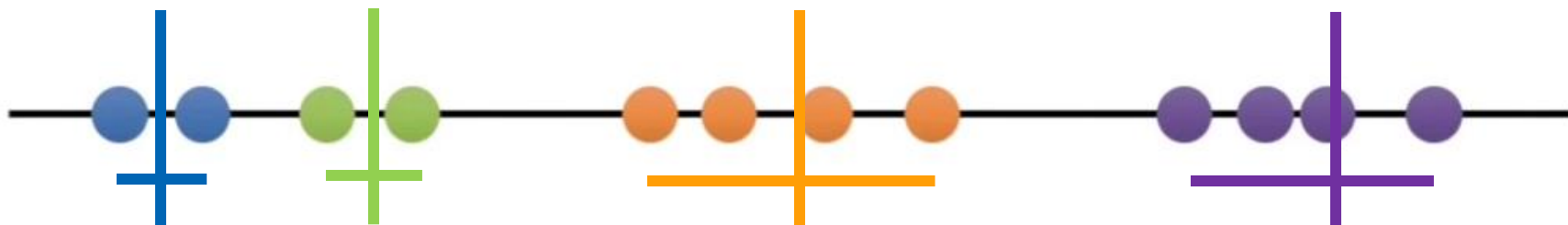


K=3



k 均值算法：肘部法则

K=4



K=1 

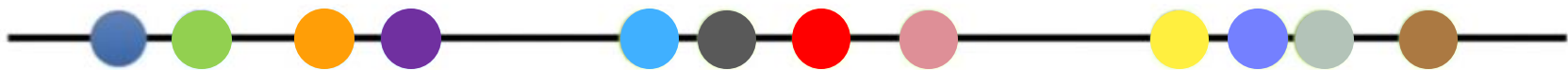
K=2 

K=3 

K=4 

k 均值算法：肘部法则

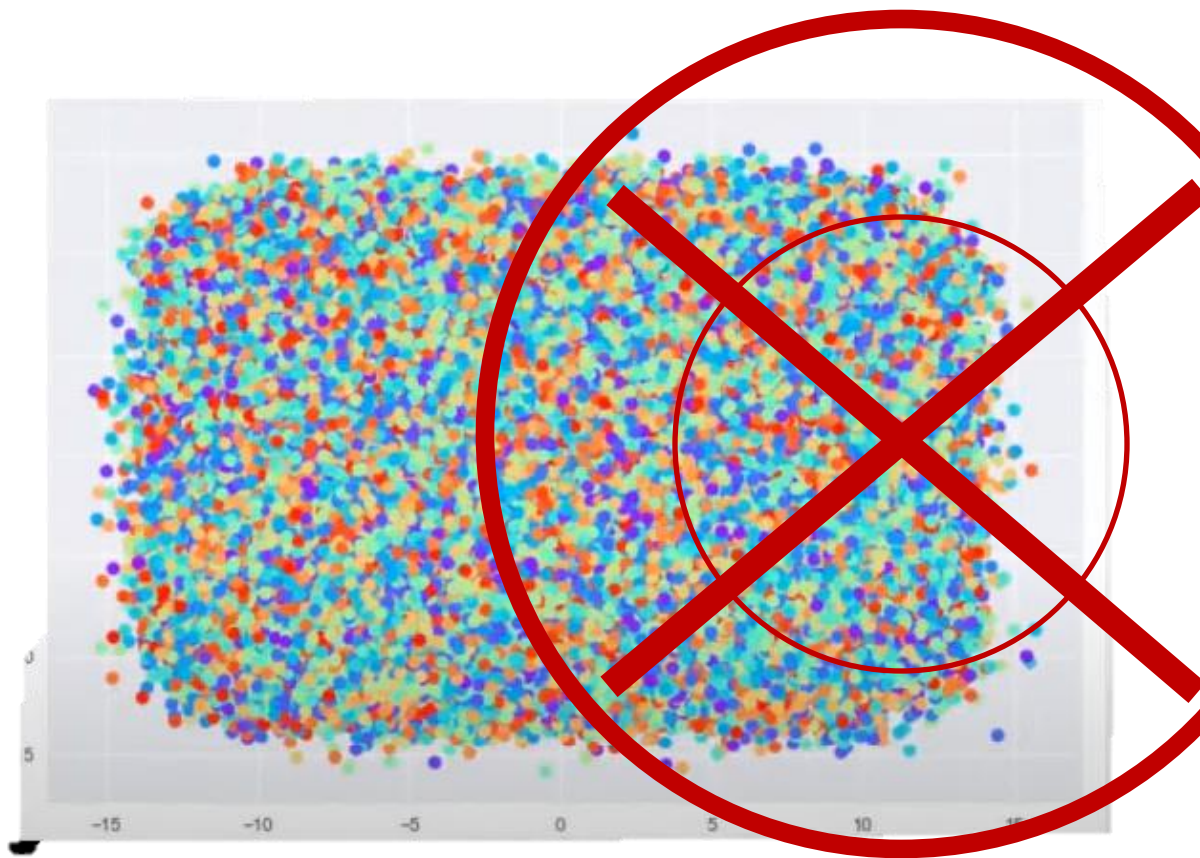
K=12



类内总变异=0

问题：这时的聚类效果怎么样呢？

k 均值算法：肘部法则



数据点的数量：

1,000,000

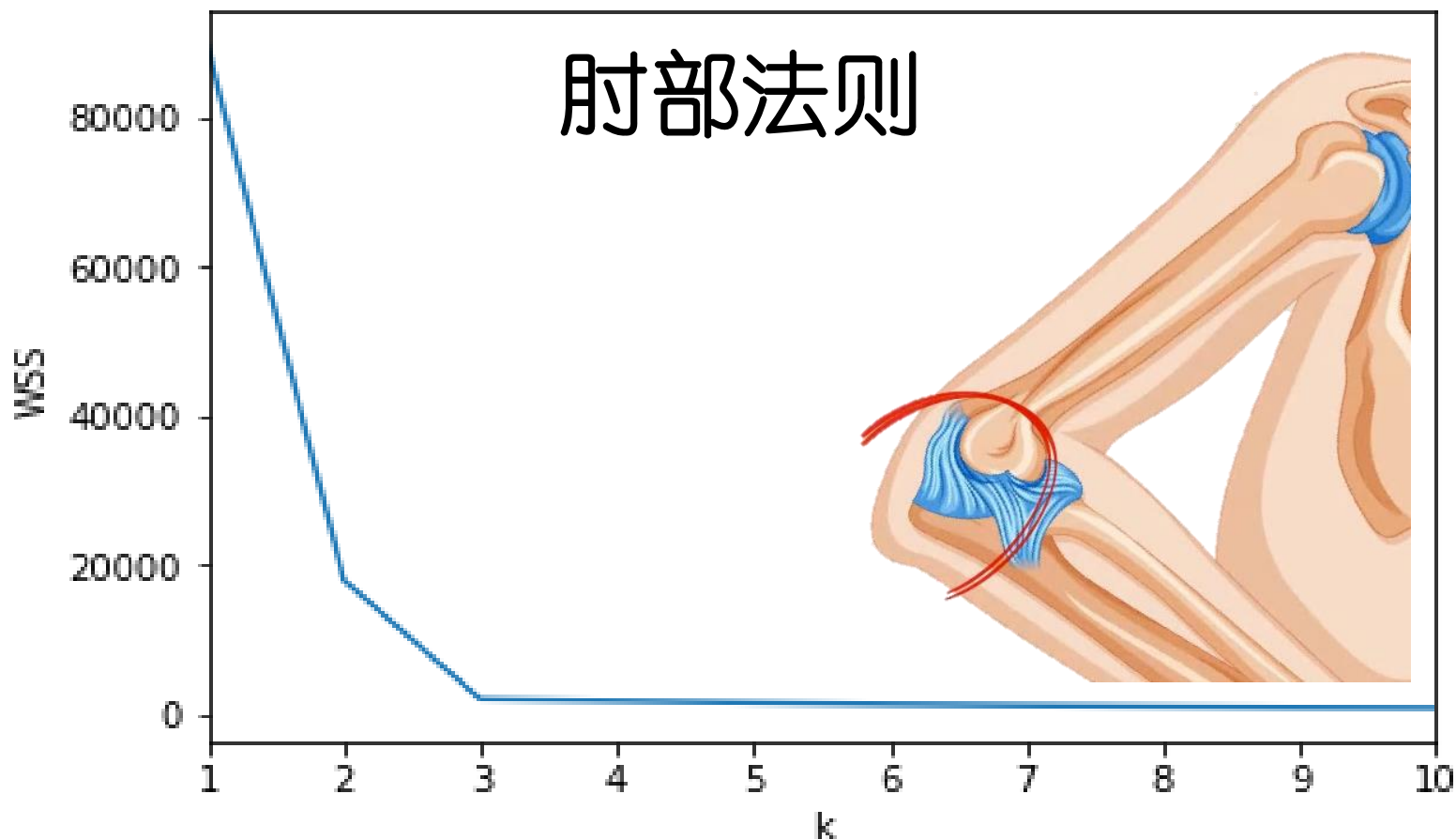
簇的数量：

1,000,000

WSS = 0 = 最小

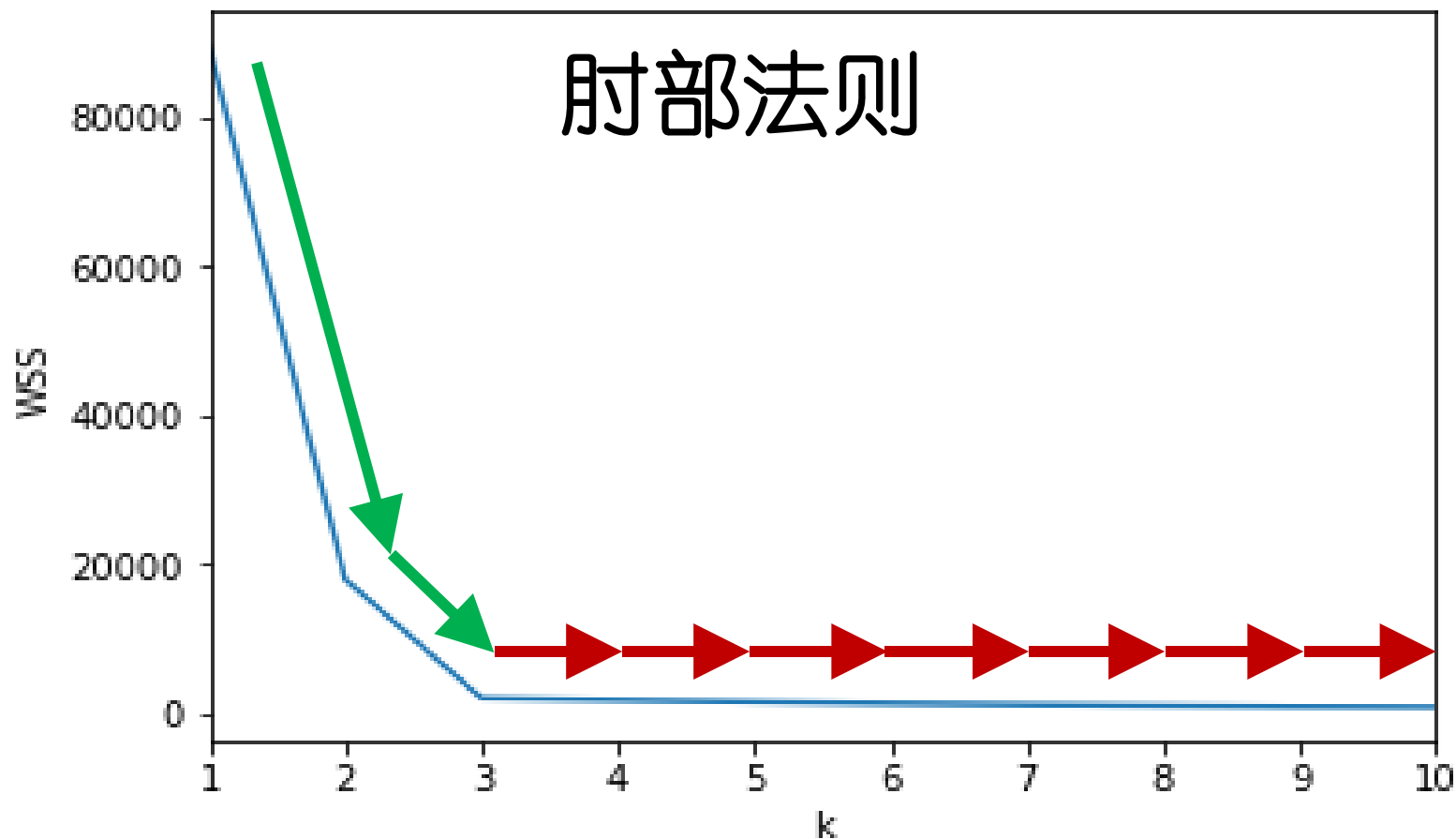
WSS：类内平方和误差，**within-cluster sum-of-squares error**

k 均值算法：肘部法则



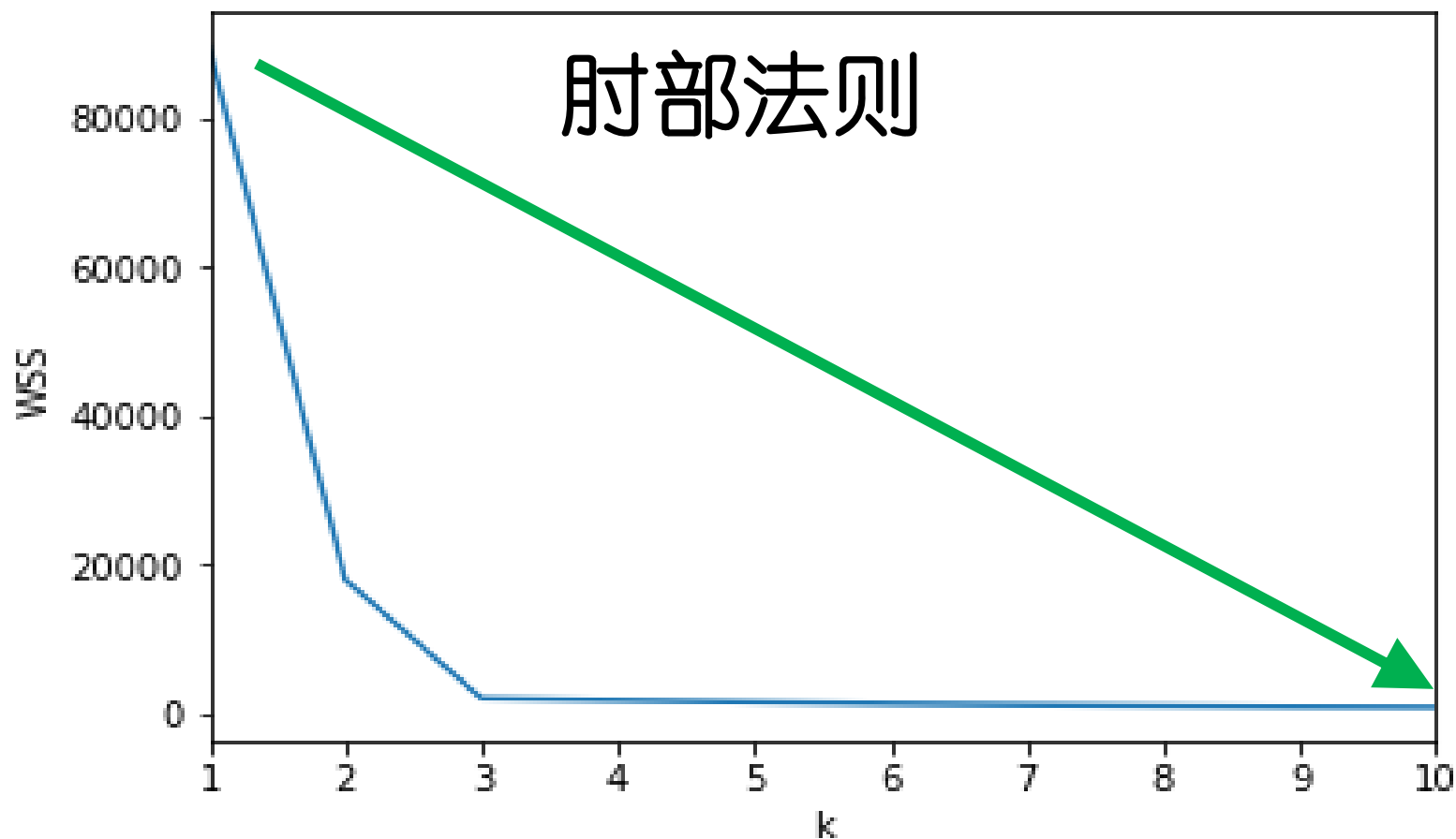
类内平方和误差 (WSS, Within-cluster Sum of Squares error)

k 均值算法：肘部法则



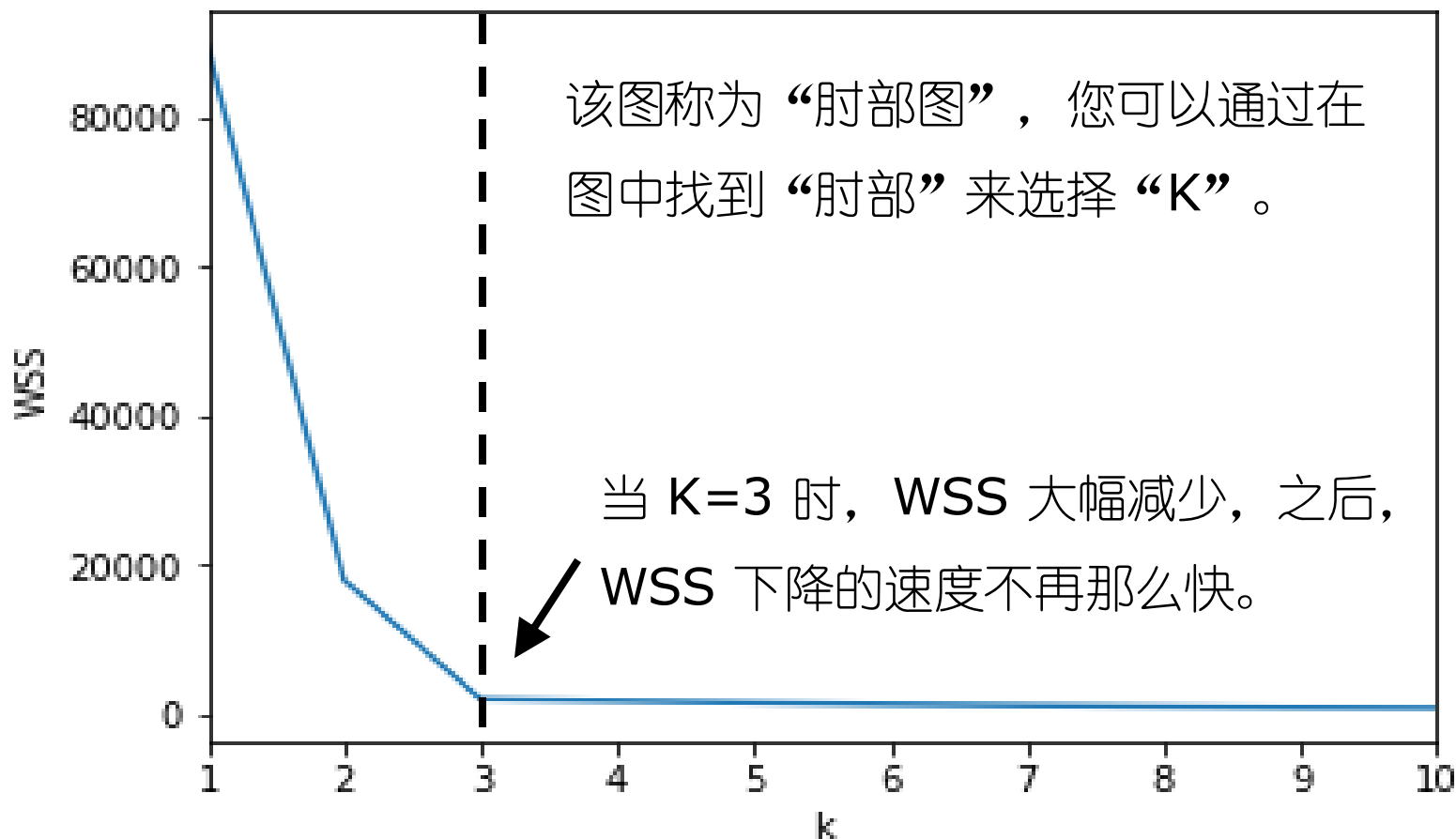
类内平方和误差 (WSS, Within-cluster Sum of Squares error)

k 均值算法：肘部法则



类内平方和误差 (WSS, Within-cluster Sum of Squares error)

k 均值算法：肘部法则

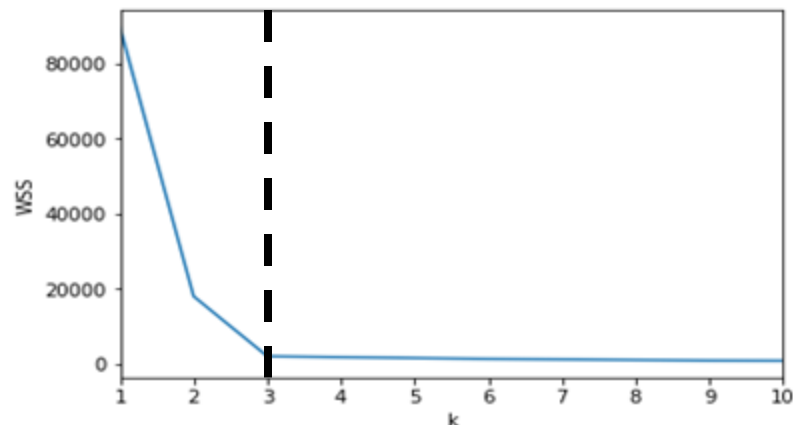
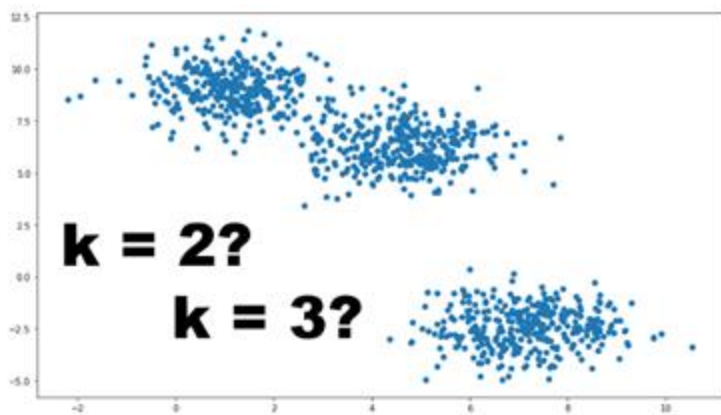


类内平方和误差 (WSS, Within-cluster Sum of Squares error)

如何选取K值：肘部法则小结

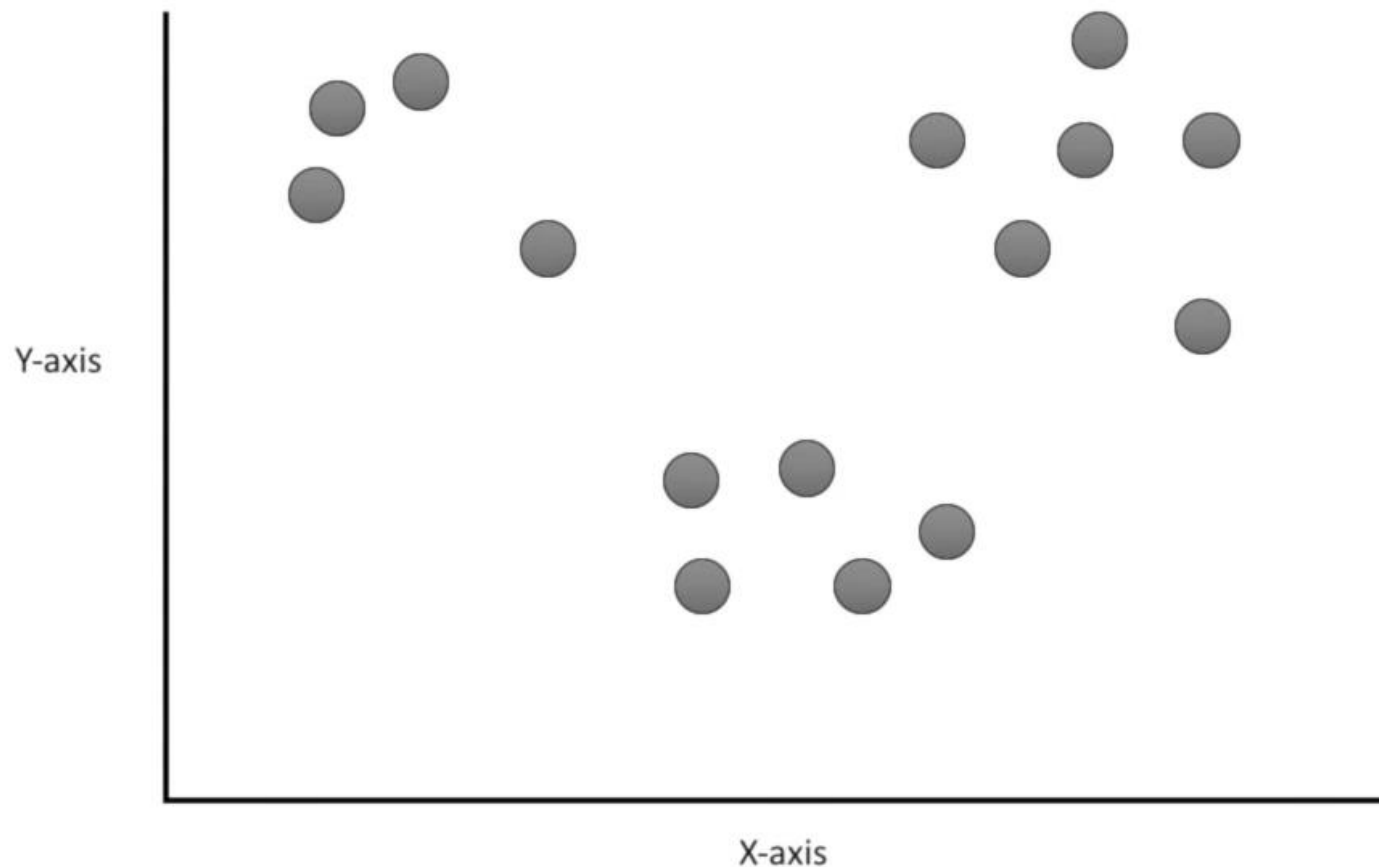
- 在 k 均值算法（K-Means）中，聚类数量应预先定义。
- 肘部法则：通过观察误差平方和（WSS）与K值的关系图来选择最佳的聚类数：

肘部图中WSS下降速率变缓的位置即为最佳的聚类数目K



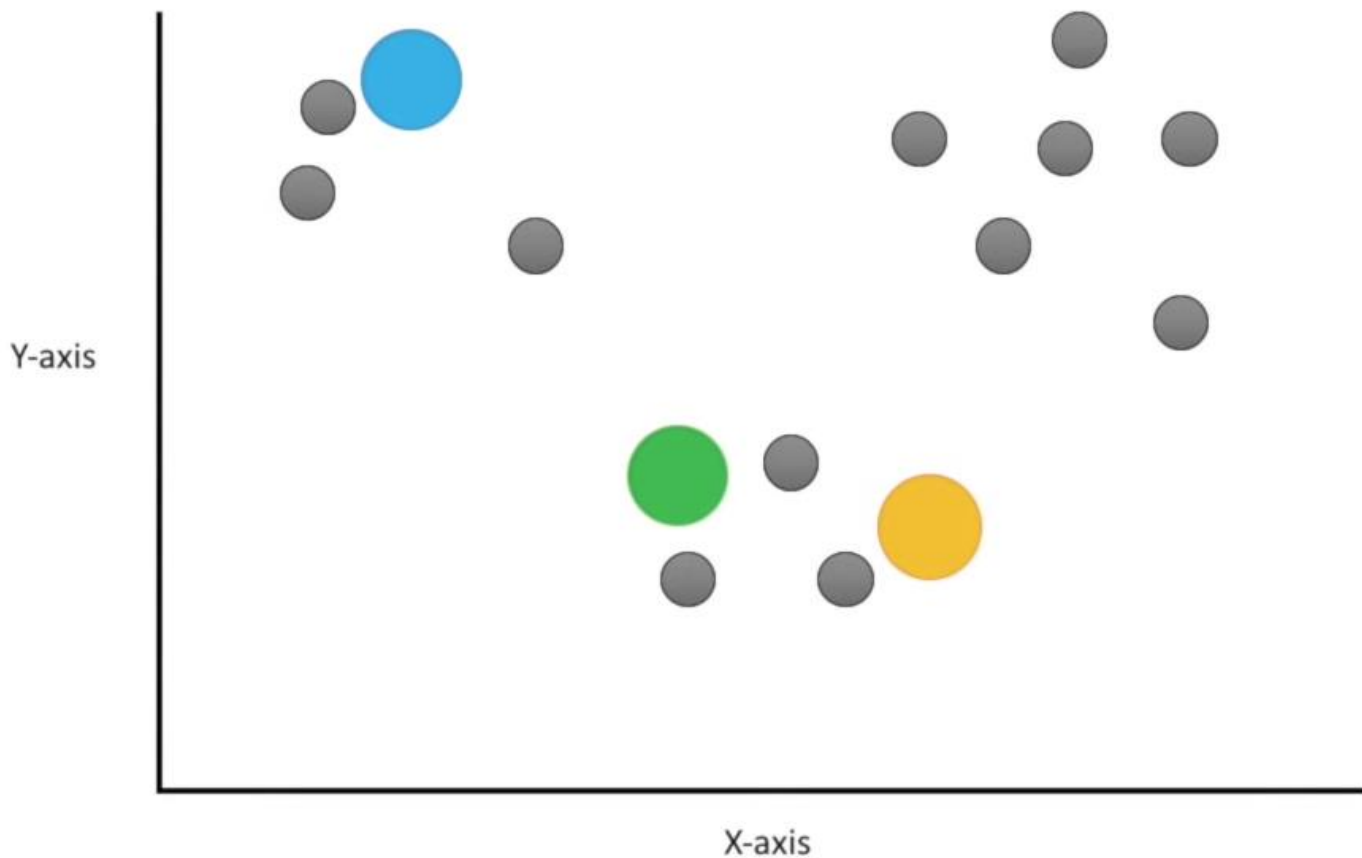
k 均值算法：例子3

问题：如果数据是二维数据的情况下，要怎么进行计算呢？



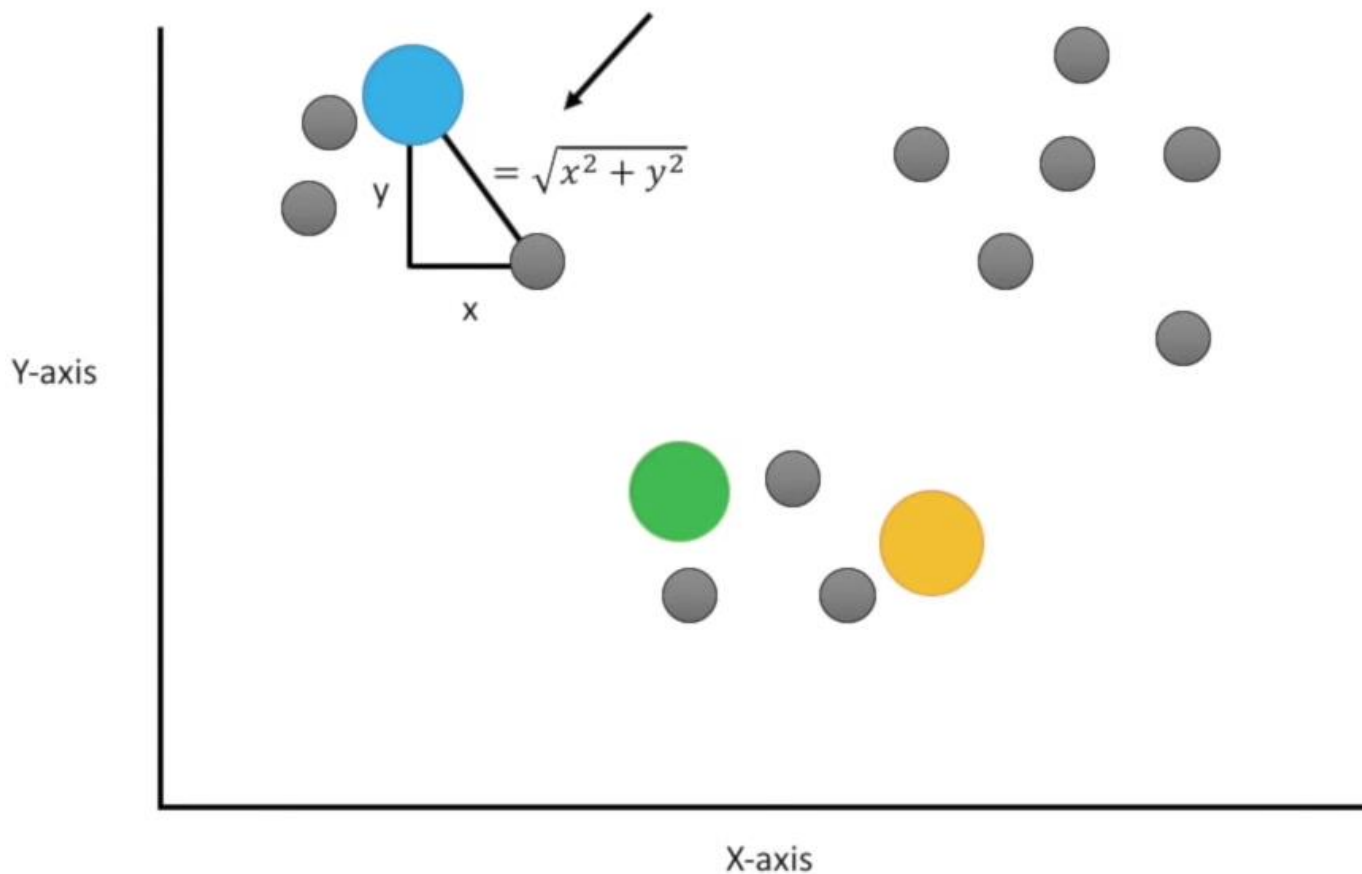
k 均值算法：例子3

就像前面例子2一样，你选择 3 个随机点作为质心.....



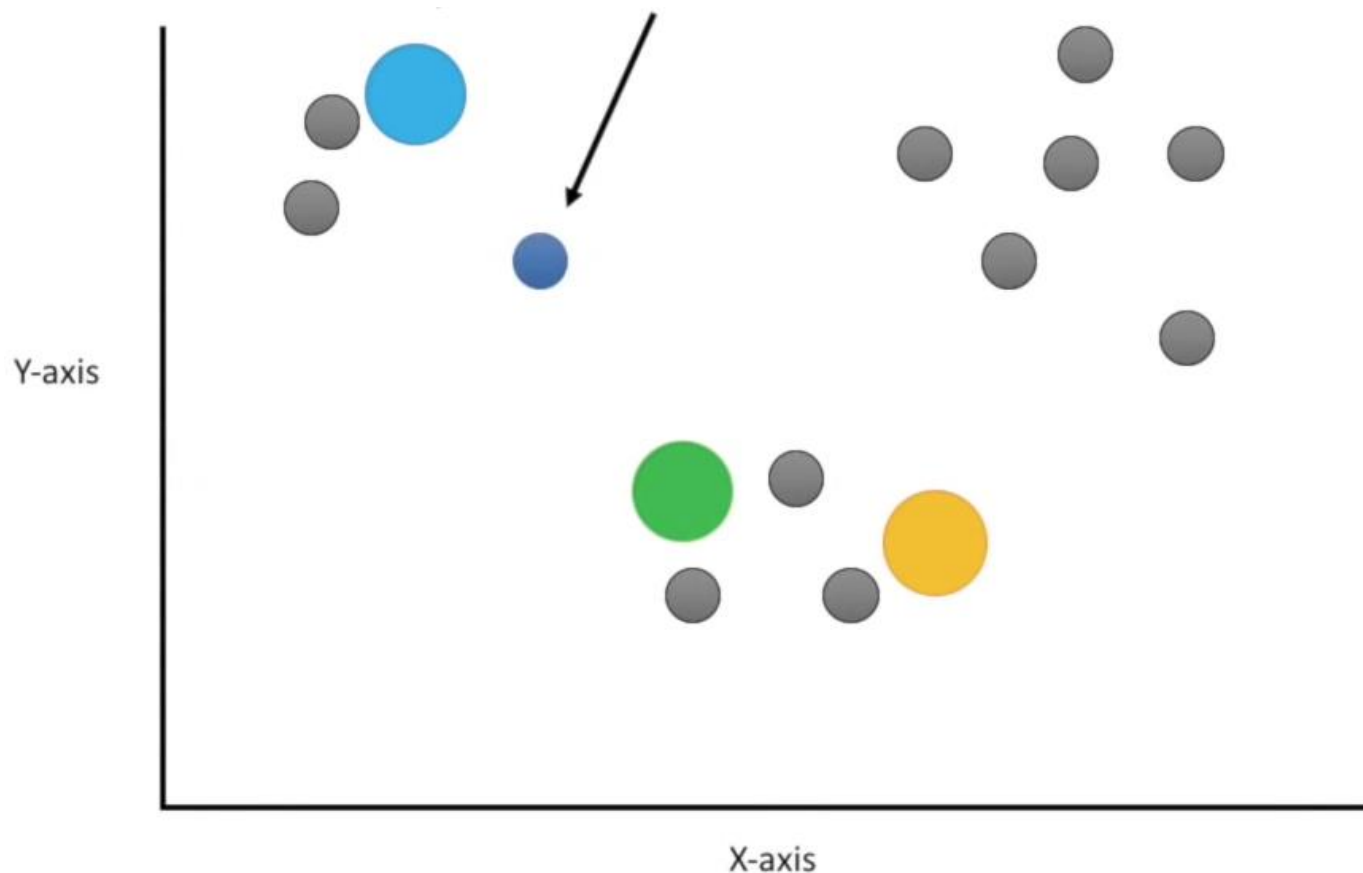
k均值算法：例子3

在二维坐标轴上，距离一般选取的是欧式距离。



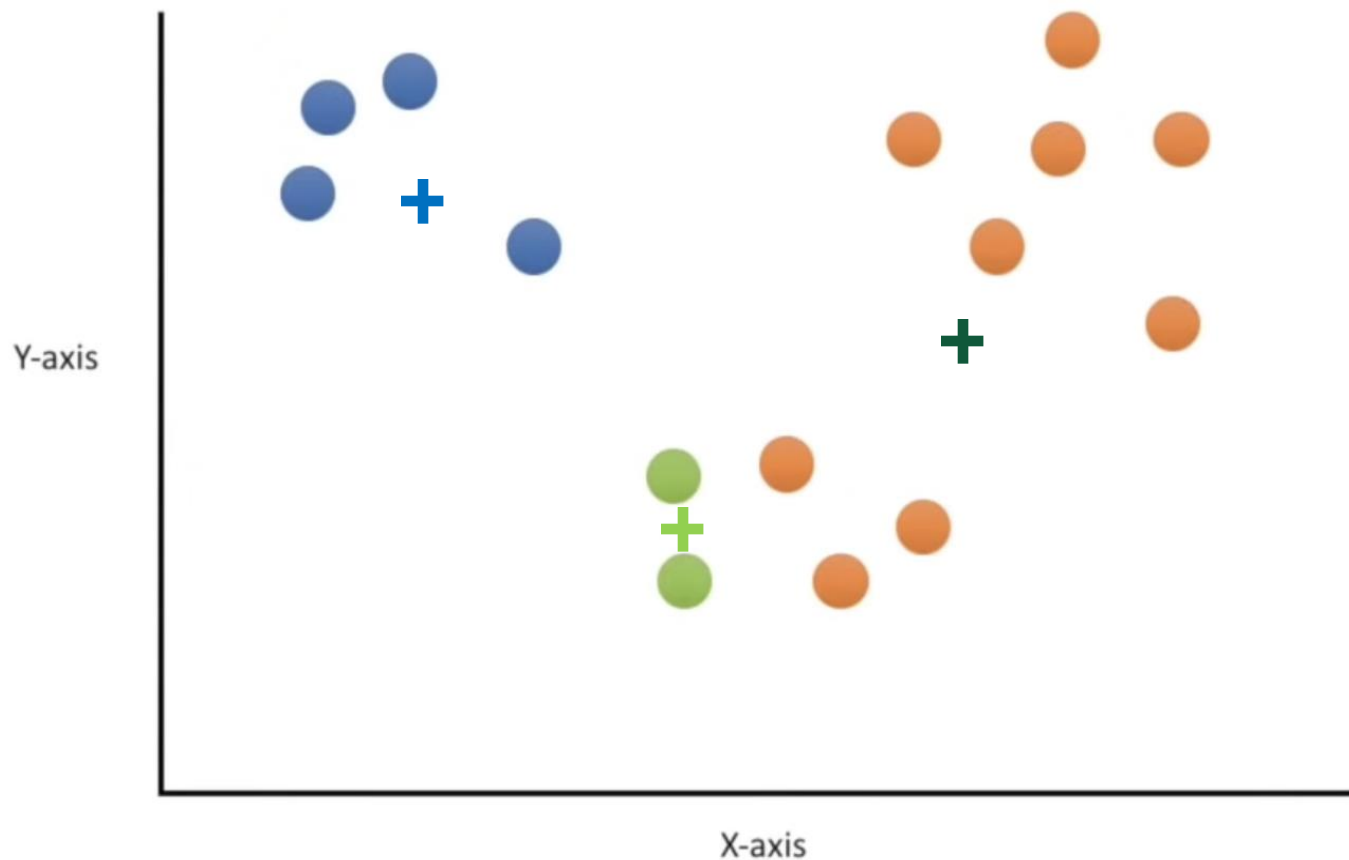
k 均值算法：例子3

就像前面例子2一样，我们将点分配到最近的簇。



k 均值算法：例子3

同样像前面例子2一样，更新每个簇的质心并重新迭代.....



大纲

- 聚类任务
- 性能度量
- 距离计算
- K均值算法
- **密度聚类（选讲）**
- 层次聚类（选讲）

密度聚类

□ 密度聚类的定义

密度聚类也称为“基于密度的聚类” (density-based clustering)。

此类算法假设聚类结构能通过样本分布的紧密程度来确定。

通常情况下，密度聚类算法从样本密度的角度来考察样本之间的可连接性，并基于可连接样本不断扩展聚类簇来获得最终的聚类结果。

接下来介绍**DBSCAN**这一密度聚类算法。

密度聚类

□ DBSCAN算法：基于一组“邻域”参数 $(\epsilon, MinPts)$ 来刻画样本分布的紧密程度。

□ 基本概念：

- ϵ 邻域：对样本 $x_j \in D$ ，其 ϵ 邻域包含样本集 D 中与 x_j 的距离不大于 ϵ 的样本；
- 核心对象：若样本 x_j 的 ϵ 邻域至少包含 $MinPts$ 个样本，则该样本点为一个核心对象；
- 密度直达：若样本 x_j 位于样本 x_i 的 ϵ 邻域中，且 x_i 是一个核心对象，则称样本 x_j 由 x_i 密度直达；
- 密度可达：对样本 x_i 与 x_j ，若存在样本序列 $p_1, p_2, \dots, p_n$ ，其中 $p_1 = x_i, p_n = x_j$ 且 p_{i+1} 由 p_i 密度直达，则该两样本密度可达；
- 密度相连：对样本 x_i 与 x_j ，若存在样本 x_k 使得两样本均由 x_k 密度可达，则称该两样本密度相连。

密度聚类

□ 一个例子

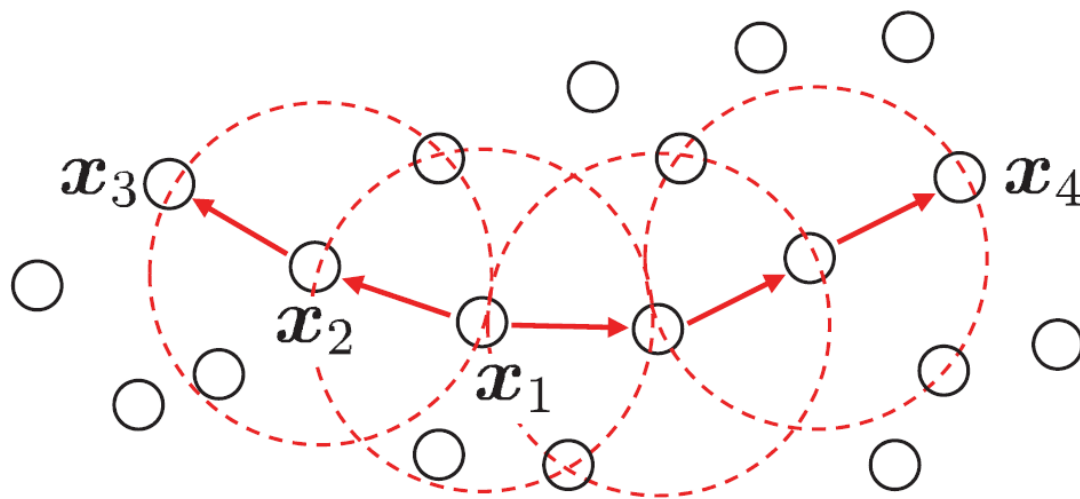
令 $MinPts = 3$ ，则
虚线显示出 ϵ 领域。

x_1 是核心对象。

x_2 由 x_1 密度直达。

x_3 由 x_1 密度可达。

x_3 与 x_4 密度相连。



密度聚类

□ 对“簇”的定义

由密度可达关系导出的最大密度相连样本集合。

□ 对“簇”的形式化描述

给定领域参数，簇是满足以下性质的非空样本子集：

连接性： $x_i \in C, x_j \in C \Rightarrow x_i$ 与 x_j 密度相连

最大性： $x_i \in C, x_i$ 与 x_j 密度可达 $\Rightarrow x_j \in C$

实际上，若 x 为核心对象，由 x 密度可达的所有样本组成的集合记为 $X = \{x' \in D \mid x' \text{ 由 } x \text{ 密度可达}\}$ ，则 X 为满足连接性与最大性的簇。

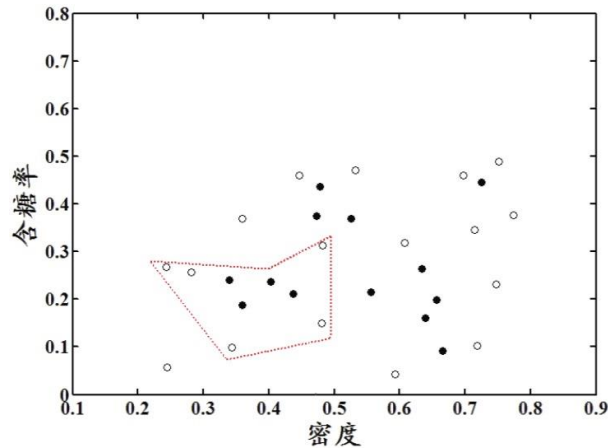
密度聚类

□ DBSCAN算法伪代码：

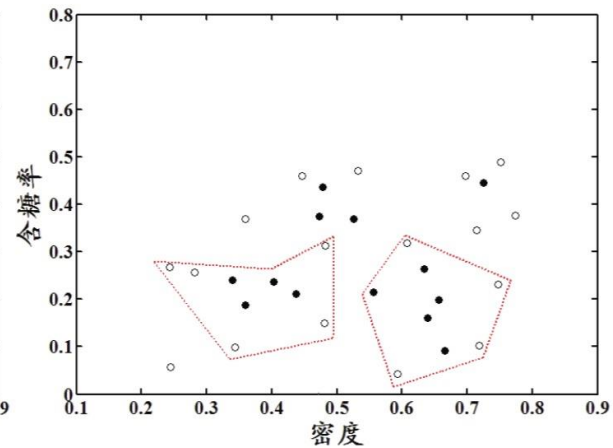
```
输入：样本集  $D = \{x_1, x_2, \dots, x_m\}$ ;  
      邻域参数  $(\epsilon, MinPts)$ .  
过程：  
1: 初始化核心对象集合：  $\Omega = \emptyset$   
2: for  $j = 1, \dots, m$  do  
3:   确定样本  $x_j$  的  $\epsilon$ -邻域  $N_\epsilon(x_j)$ ;  
4:   if  $|N_\epsilon(x_j)| \geq MinPts$  then  
5:     将样本  $x_j$  加入核心对象集合：  $\Omega = \Omega \cup \{x_j\}$   
6:   end if  
7: end for  
8: 初始化聚类簇数：  $k = 0$   
9: 初始化未访问样本集合：  $\Gamma = D$   
10: while  $\Omega \neq \emptyset$  do  
11:   记录当前未访问样本集合：  $\Gamma_{old} = \Gamma$ ;  
12:   随机选取一个核心对象  $o \in \Omega$ , 初始化队列  $Q = \langle o \rangle$ ;  
13:    $\Gamma = \Gamma \setminus \{o\}$ ;  
14:   while  $Q \neq \emptyset$  do  
15:     取出队列  $Q$  中的首个样本  $q$ ;  
16:     if  $|N_\epsilon(q)| \geq MinPts$  then  
17:       令  $\Delta = N_\epsilon(q) \cap \Gamma$ ;  
18:       将  $\Delta$  中的样本加入队列  $Q$ ;  
19:        $\Gamma = \Gamma \setminus \Delta$ ;  
20:     end if  
21:   end while  
22:    $k = k + 1$ , 生成聚类簇  $C_k = \Gamma_{old} \setminus \Gamma$ ;  
23:    $\Omega = \Omega \setminus C_k$   
24: end while  
25: return 簇划分结果  
输出：簇划分  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ 
```

密度聚类

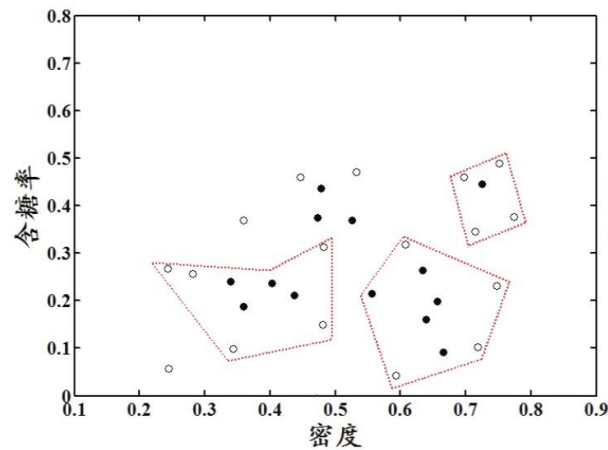
□ 聚类效果：



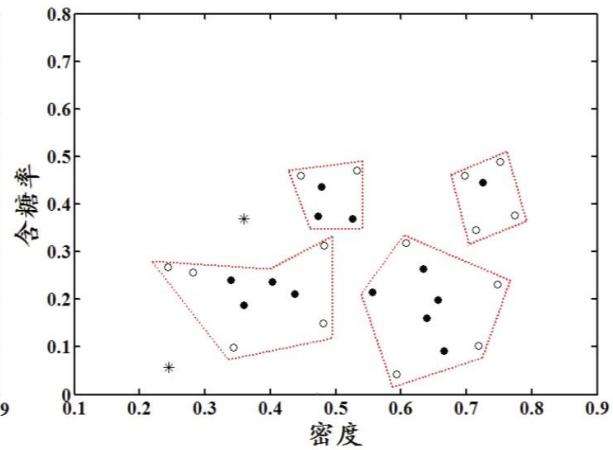
(a) 生成聚类簇 C_1



(b) 生成聚类簇 C_2



(c) 生成聚类簇 C_3



(d) 生成聚类簇 C_4

大纲

- 聚类任务
- 性能度量
- 距离计算
- K均值算法
- 密度聚类（选讲）
- 层次聚类（选讲）

层次聚类

□ 层次聚类试图在不同层次对数据集进行划分，从而形成树形的聚类结构。数据集划分既可采用“自底向上”的聚合策略，也可采用“自顶向下”的分拆策略。

□ **AGNES**算法（自底向上的层次聚类算法）

首先，将样本中的每一个样本看做一个初始聚类簇，然后在算法运行的每一步中找出距离最近的两个聚类簇进行合并，该过程不断重复，直到达到预设的聚类簇的个数。

这里两个聚类簇 C_i 和 C_j 的距离，可以有3种度量方式。

层次聚类

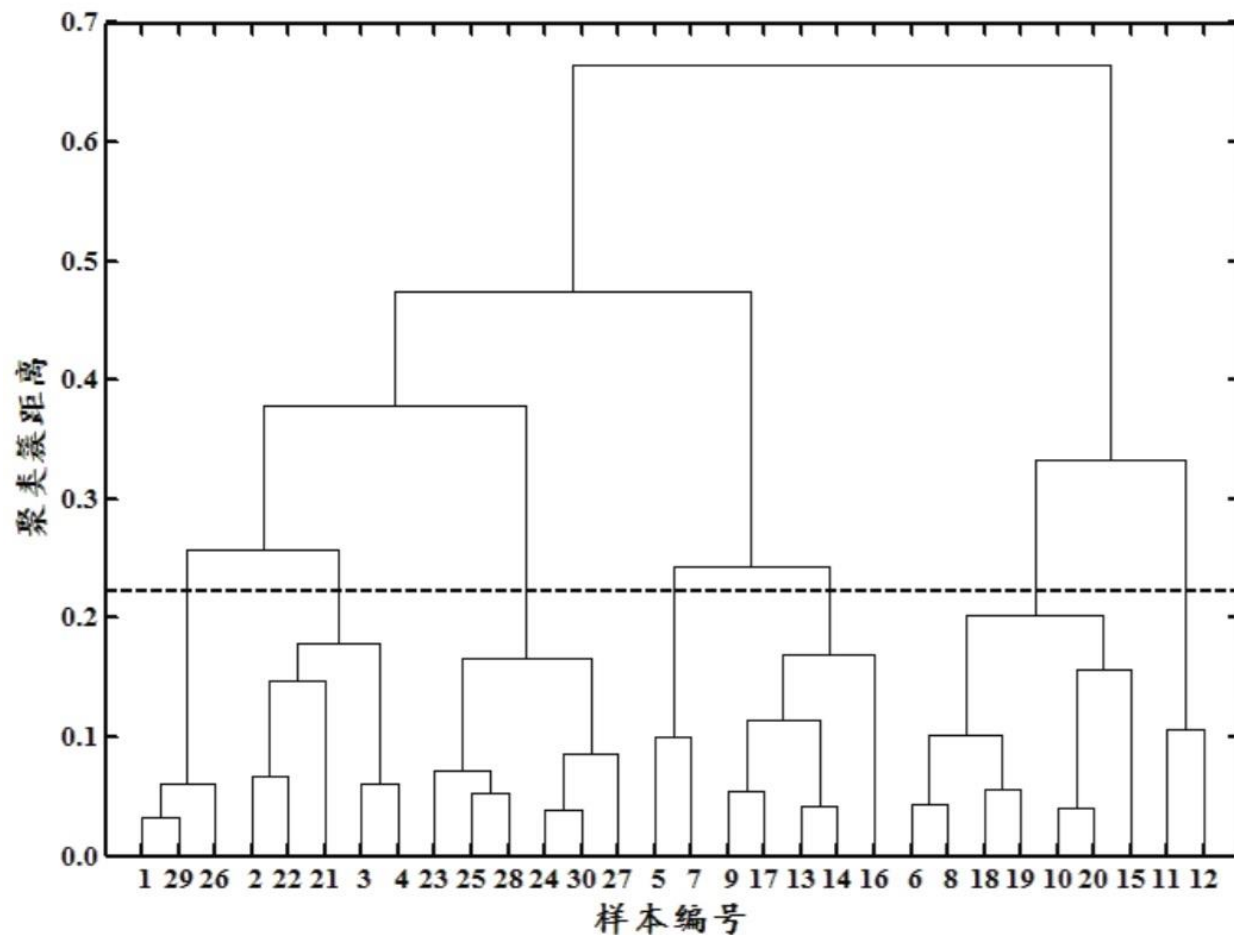
最小距离:
$$d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$$

最大距离:
$$d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$$

平均距离:
$$d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$$

层次聚类 - 树状图

□ AGNES算法树状图：



层次聚类 – AGNES算法

□ AGNES算法伪代码：

输入：样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇距离度量函数 $d \in \{d_{\min}, d_{\max}, d_{\text{avg}}\}$;
聚类簇数 k .

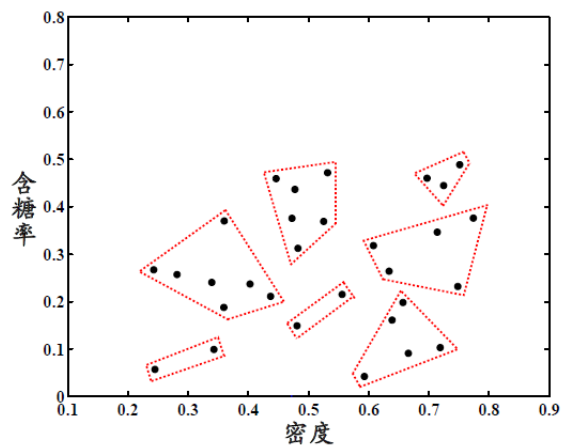
过程：

```
1: for  $j = 1, \dots, m$  do
2:    $C_j = \{x_j\}$ 
3: end for
4: for  $i = 1, \dots, m$  do
5:   for  $j = i, \dots, m$  do
6:      $M(i, j) = d(C_i, C_j)$ ;
7:      $M(j, i) = M(i, j)$ 
8:   end for
9: end for
10: 设置当前聚类簇个数:  $q = m$ 
11: while  $q > k$  do
12:   找出距离最近的两个聚类簇  $(C_{i^*}, C_{j^*})$ ;
13:   合并  $(C_{i^*}, C_{j^*})$ :  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
14:   for  $j = j^* + 1, \dots, q$  do
15:     将聚类簇  $C_j$  重编号为  $C_{j-1}$ 
16:   end for
17:   删除距离矩阵  $M$  的第  $j^*$  行与第  $j^*$  列;
18:   for  $j = 1, \dots, q - 1$  do
19:      $M(i^*, j) = d(C_{i^*}, C_j)$ ;
20:      $M(j, i^*) = M(i^*, j)$ 
21:   end for
22:    $q = q - 1$ 
23: end while
24: return 簇划分结果
```

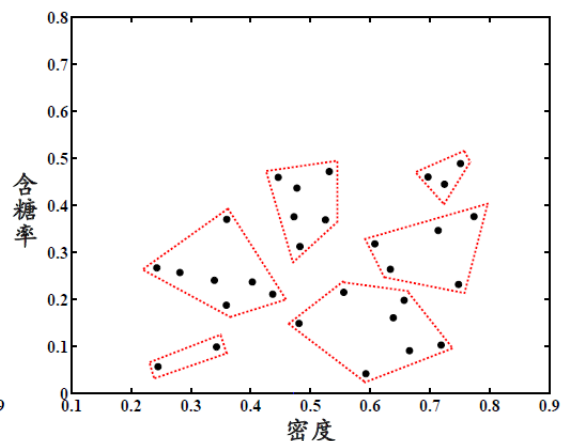
输出：簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

层次聚类

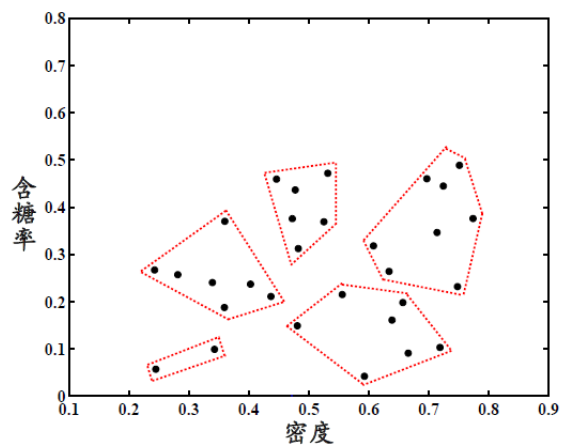
AGNES算法聚类效果：



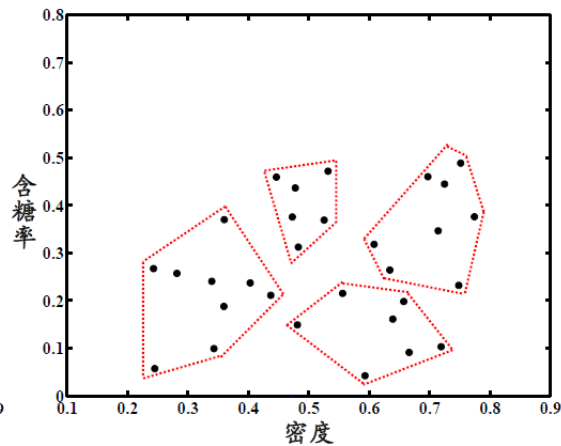
(a) 聚类簇数 $k = 7$



(b) 聚类簇数 $k = 6$



(c) 聚类簇数 $k = 5$



(d) 聚类簇数 $k = 4$